



National University of Computer and Emerging Sciences, Lahore



StockSense

Muhammad Usman 23L-0693 BS(CS)

Muhammad Rizwan Arshad 23L-0755 BS(CS)

Hayyan Haider 23L-0727 BS(CS)

Supervisor: Mr. Saifullah Tanvir

Final Year Project

October 10, 2026

Anti-Plagiarism Declaration

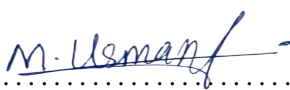
This is to declare that the above publication was produced under the:

Title: StockSense

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date:

Name: Muhammad Usman

Signature: 

Name: Muhammad Rizwan Arshad

Signature: 

Name: Hayyan Haider

Signature: 

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

Retailers lose money daily when a product, like clothing, footwear or any other physical product with variants, sizes or stores, is out of stock or dead stock. For large chains, the solution is enterprise forecasting systems such as SAP and Blue Yonder, which small businesses can't afford to purchase or maintain. This project aims to develop a multi-tenant SaaS application using a general purpose forecasting engine which can be applicable to any stock-keeping business, called StockSense. It features a free point of sale (POS) solution, location- and variant-level demand forecasting, a markdown/discount optimizer and stock-redistribution suggestions are available as paid subscription options. However, StockSense differs from current tools in that it recommends inter-branch redistribution as its top priority, and alerts the officer with an alert early enough that they can act on it before it becomes a "dead stock", not after it has already died. Existing businesses using a POS or spreadsheets can sign up with uploading their sales reports.

Executive Summary

Retailers face two opposing problems each day that cost them revenue. Dead stock is stored for months, requiring capital and space, while stock outs result in loss of sales as the item the customer is interested in is not available, and less popular items are increasing in quantity. This affects any retailer that manages physical inventory across variants, sizes, or store locations, from clothing and footwear shops to many other kinds of stores. The issue is most severely felt in smaller businesses because restocking decisions are typically made based on instinct. No systematic understanding of what items at what locations meet demand; a manager reorders what he or she thinks sold well.

Large retailers solve this problem by using enterprise demand-forecasting platforms like SAP and Blue Yonder, but these are also expensive and designed for those with an established data team and ERP system. Larger retailers may be able to add another forecasting service to the top of their ERP. Small retailers using basic POS systems or spreadsheets lack an equivalent solution. Even the lighter of inventory software solutions for small businesses usually provide aggregate forecasts (demand predicted per product, not per size or variant), reorder points (alerts only when stock runs low), and slow-mover reports (lists items that already sold poorly), and work with each store's inventory as a separate entity.

To fill this gap, StockSense is a multi-tenant SaaS platform. It is based on a general purpose forecasting engine which can be used for any business where stocks are kept, not just a particular product category. The platform enables small retailers to get a free POS that allows them to record sales and keep an eye on stock and brings their data to the platform, meeting a day-to-day requirement. In addition to that data, a paid subscription opens up the location and variant level demand forecasts and provides an automated markdown/discount optimizer and stock-redistribution recommendations. Companies that already use another point-of-sale system or spreadsheets can join without having to change anything, simply by uploading their previous sales data in CSV or Excel format. In both, owners and managers access their past sales data, current prediction and suggestions for restocking via a simple web dashboard accessible to non-technical users.

StockSense differs from existing solutions in three main ways. First, it prioritizes redistribution. Existing tools generate recommendations for each store independently and mention stock transfers only as one tactic among several. If a business has 2, 3 or more branches, it is almost always cheaper and faster to move unsold stock from a branch where it is not selling to a branch where it is sold, than to sell the same stock at a discount or place a new order – StockSense identifies this as a first class recommendation, ahead of markdowns or reordering.

Second, its intervention is preventive rather than reactive. Existing technologies see stock as dead when

it is too old to be a viable option, and then offer write-off or discount. The stock problem now becomes a timing problem because StockSense markdown optimizer detects a stock as it is coming to a slow-down and alerts the manager in time to take action. Thirdly, all recommendations are explained. Every suggestion is accompanied by a simple explanation of why it is warranted and an estimate of the cost, so that a business owner can better understand and be confident in the suggestion rather than blindly following it as a black box.

Preliminary work has included a review of the demand-forecasting and small-business inventory literature, surveying the available enterprise and small-business platforms, and identifying available public retail sales datasets for initial model development and validation. We could not find any platform that offers a free POS system with a possibly paid dead-stock-reduction layer to help small retailers, making StockSense dead-stock-reduction layer freemium seem to be the only one of its kind. The accuracy of the forecast will be judged using standard error metric and potential reduction of dead stock will be simulated with historical data.

To summarize, StockSense was designed to help a small business leverage the sales data that they already gather, but never analyze, to make actionable and inexpensive decisions to decrease dead stock and recover lost sales. It makes demand forecasting and inventory optimization, something traditionally only large retailers have access to, available to small businesses.

Table of Contents

List of Figures	xii
------------------------	------------

List of Tables	xiii
-----------------------	-------------

1 Introduction	1
1.1 Purpose of this Document	1
1.2 Intended Audience	1
1.3 Definitions, Acronyms, and Abbreviations	1
1.4 Conclusion	2
2 Project Vision	3
2.1 Problem Domain Overview	3
2.2 Problem Statement	3
2.3 Problem Elaboration	3
2.4 Goals and Objectives	4
2.5 Project Scope	4
2.6 Sustainable Development Goal (SDG)	5
2.6.1 SDG-8 (Decent Work and Economic Growth)	5
2.6.2 SDG-9 (Industry, Innovation, and Infrastructure)	5
2.7 Constraints	5
2.8 Business Opportunity	5
2.9 Stakeholders Description / User Characteristics	6
2.9.1 Key High-Level Goals and Problems of Stakeholders	6
2.10 Conclusion	6
3 Literature Review / Related Work	7
3.1 Definitions and Acronyms	7
3.1.1 Definitions	7
3.1.2 Acronyms	7

3.2	Detailed Literature Review	7
3.2.1	Exploring the Use of Deep Neural Networks for Sales Forecasting in Fashion Retail	7
3.2.2	Clearance Pricing Optimization for a Fast-Fashion Retailer	8
3.2.3	Inventory Models with Lateral Transshipments: A Review	8
3.2.4	Forecasting Across Time Series Databases Using Recurrent Neural Networks on Groups of Similar Series: A Clustering Approach	9
3.2.5	M5 Accuracy Competition: Results, Findings, and Conclusions	9
3.2.6	Demand Forecasting for Improved Inventory Management in Small and Medium- Sized Businesses	10
3.2.7	Retail Forecasting: Research and Practice	10
3.2.8	Principles and Algorithms for Forecasting Groups of Time Series: Locality and Globality	11
3.2.9	Reducing the Cost of Demand Uncertainty Through Accurate Response to Early Sales	11
3.2.10	Sales Forecasts in Clothing Industry: The Key Success Factor of the Supply Chain Management	12
3.2.11	Predictive Big Data Analytics for Supply Chain Demand Forecasting: Methods, Applications, and Research Opportunities	12
3.2.12	Forecasting: Theory and Practice	12
3.2.13	From Predictive to Prescriptive Analytics	13
3.2.14	Stop Explaining Black Box Machine Learning Models for High Stakes Deci- sions and Use Interpretable Models Instead	13
3.2.15	Desperately Seeking Shelf Availability: An Examination of the Extent, the Causes, and the Efforts to Address Retail Out-of-Stocks	13
3.3	Literature Review Summary Table	14
3.4	Related Work	15
3.4.1	Blue Yonder	15
3.4.2	Nextail	16
3.4.3	Shopify POS (with Stocky)	16
3.4.4	Lightspeed Retail	16
3.4.5	Odoo	16
3.5	Related Work Summary Table	17
3.6	Conclusion	17

3.7	Selected Model	17
4	Software Requirement Specifications	18
4.1	List of Features	18
4.2	Functional Requirements	18
4.2.1	Functional Requirements of the Business Owner	18
4.2.2	Functional Requirements of the Store Manager	18
4.2.3	Functional Requirements of the System	18
4.3	Quality Attributes	19
4.3.1	Reliability	19
4.3.2	Maintainability	19
4.3.3	Usability	19
4.3.4	Correctness	19
4.3.5	Efficiency	19
4.3.6	Flexibility	19
4.4	Non-Functional Requirements	19
4.4.1	Performance	19
4.4.2	Reliability	19
4.4.3	Security	20
4.4.4	Compatibility	20
4.5	Assumptions	20
4.6	Use Cases	20
4.7	Hardware and Software Requirements	27
4.7.1	Hardware Requirements	27
4.7.2	Software Requirements	28
4.8	Graphical User Interface	28
4.8.1	Login Page	28
4.8.2	Signup Page	29
4.8.3	Dashboard	30
4.8.4	Point of Sale	31
4.8.5	Upload Sales Report	32
4.8.6	Inventory	33
4.8.7	Recommendations	34
4.9	Database Design	35
4.9.1	ER Diagram	35

4.9.2	Data Dictionary	36
4.10	Risk Analysis	37
4.11	Conclusion	37
5	Proposed Approach and Methodology	38
5.1	Overall Pipeline	38
5.2	Data Ingestion	38
5.3	Data Validation, Cleaning, and Normalization	38
5.4	Demand Series Construction	39
5.5	Feature Engineering	39
5.6	Demand Forecasting Model	39
5.6.1	Global Gradient-Boosted Model	39
5.6.2	Loss Function and Forecast Horizon	39
5.6.3	Training and Tuning	40
5.6.4	Statistical Baselines	40
5.6.5	Initial Training Period	40
5.6.6	Forecast Uncertainty	40
5.7	Recommendation Engine and Stock Redistribution	40
5.8	Markdown Optimization	41
5.9	Explainable Recommendations	41
5.10	Forecast and Recommendation Evaluation	42
5.10.1	Data	42
5.10.2	Protocol	42
5.10.3	Metrics	42
5.10.4	Experiments	42
5.10.5	Recommendation Backtest	42
5.11	Iterative Development	43
5.12	Conclusion	43
6	High-Level and Low-Level Design	44
6.1	System Overview	44
6.1.1	Tenant and User Management	44
6.1.2	POS and Inventory Management	44
6.1.3	Data Ingestion and Cleaning	44
6.1.4	Demand Forecasting	44

6.1.5	Stock Redistribution and Reorder Recommendations	44
6.1.6	Markdown Optimization	44
6.1.7	Explainable Recommendations	44
6.1.8	Dashboard and Reporting	44
6.2	Design Considerations	45
6.2.1	Assumptions and Dependencies	45
6.2.2	General Constraints	45
6.2.3	Goals and Guidelines	46
6.2.4	Development Methods	46
6.3	System Architecture	46
6.3.1	Subsystem Architecture	47
6.4	Architectural Strategies	48
6.4.1	Multi-Tenant Architecture	48
6.4.2	Modular Architecture	48
6.4.3	Programming Languages and Frameworks	48
6.4.4	Database	48
6.4.5	Background Processing	48
6.4.6	Error Detection and Recovery	48
6.5	Domain Model/Class Diagram	48
6.6	Sequence Diagrams	50
6.6.1	UC-01: Register Business	50
6.6.2	UC-02: Login	50
6.6.3	UC-03: Manage Stores	51
6.6.4	UC-04: Manage Store Managers	52
6.6.5	UC-05: Manage Products and Variants	53
6.6.6	UC-06: Adjust Inventory	54
6.6.7	UC-07: Record Sale (POS)	55
6.6.8	UC-08: Upload Historical Sales Data	56
6.6.9	UC-09: View Dashboard and Alerts	57
6.6.10	UC-10: View Demand Forecast	58
6.6.11	UC-11: View and Act on Redistribution Recommendation	59
6.6.12	UC-12: View Markdown Recommendation	60
6.6.13	UC-13: View Reorder Recommendation	61
6.6.14	UC-14: Manage Subscription	62

6.6.15	UC-15: Manage Tenants	63
6.7	Policies and Tactics	64
6.7.1	Tools to be Used	64
6.7.2	Coding Guidelines and Conventions	65
6.7.3	User Interface	65
6.7.4	Extensibility	65
6.7.5	Tenant Data Isolation and Security	65
6.7.6	Data Validation	65
6.7.7	Explainable Recommendations	65
6.7.8	System Testing	65
6.7.9	Software Maintenance	66
6.7.10	Algorithms	66
6.8	Conclusion	66

List of Figures

4.1	Login Page	28
4.2	Signup Page	29
4.3	Dashboard	30
4.4	Point of Sale	31
4.5	Upload Sales Report	32
4.6	Inventory	33
4.7	Recommendations	34
4.8	ER Diagram	35
6.1	System Architecture Diagram	47
6.2	Domain Model / Class Diagram	49
6.3	Sequence Diagram: Register Business	50
6.4	Sequence Diagram: Login	51
6.5	Sequence Diagram: Manage Stores	52
6.6	Sequence Diagram: Manage Store Managers	53
6.7	Sequence Diagram: Manage Products and Variants	54
6.8	Sequence Diagram: Adjust Inventory	55
6.9	Sequence Diagram: Record Sale (POS)	56
6.10	Sequence Diagram: Upload Historical Sales Data	57
6.11	Sequence Diagram: View Dashboard and Alerts	58
6.12	Sequence Diagram: View Demand Forecast	59
6.13	Sequence Diagram: View and Act on Redistribution Recommendation	60
6.14	Sequence Diagram: View Markdown Recommendation	61
6.15	Sequence Diagram: View Reorder Recommendation	62
6.16	Sequence Diagram: Manage Subscription	63
6.17	Sequence Diagram: Manage Tenants	64

List of Tables

2.1 Stakeholders of StockSense	6
3.1 Summary of Literature Review	14
3.2 Summary of Related Work	17
4.1 UC-01: Register Business	20
4.2 UC-02: Login	21
4.3 UC-03: Manage Stores	21
4.4 UC-04: Manage Store Managers	22
4.5 UC-05: Manage Products and Variants	22
4.6 UC-06: Adjust Inventory	23
4.7 UC-07: Record Sale (POS)	23
4.8 UC-08: Upload Historical Sales Data	24
4.9 UC-09: View Dashboard and Alerts	24
4.10 UC-10: View Demand Forecast	25
4.11 UC-11: View and Act on Redistribution Recommendation	25
4.12 UC-12: View Markdown Recommendation	26
4.13 UC-13: View Reorder Recommendation	26
4.14 UC-14: Manage Subscription	27
4.15 UC-15: Manage Tenants	27
4.16 Data Dictionary for Tenant Table	36
4.17 Data Dictionary for Store Table	36
4.18 Data Dictionary for Product Variant Table	36
4.19 Data Dictionary for Sale Table	36
4.20 Data Dictionary for Recommendation Table	37

Chapter 1 Introduction

Retailers need the right products in the right quantities and locations, but small and medium-sized businesses often rely on spreadsheets, basic POS systems, or manual records without systematically analyzing sales data. This can cause stock outs and dead stock, particularly when demand for different products, sizes and locations is different in different branches. Large retailers have enterprise forecasting and inventory optimization tools, but these are expensive and difficult to use. StockSense is proposed as a multi-tenant SaaS solution that makes data-driven inventory management more accessible through a free POS system, demand forecasting, inter-branch stock redistribution, markdown optimization, and explainable recommendations. StockSense leverages existing sales data to enable retailers to detect inventory risks early and take action before products become unsold stock or cause stock outs.

1.1 Purpose of this Document

This document provides a detailed description of the StockSense Final Year Project (FYP), which aims to develop a multi-tenant SaaS platform for retail inventory management and demand forecasting. It provides small and medium-sized retailers with an easy-to-use solution to transform POS and historical sales data into actionable inventory decisions. The document covers the project's problem domain, objectives, scope, requirements, methodology, system design, and implementation, including demand forecasting, inventory redistribution, markdown recommendations, and explainable decision support.

1.2 Intended Audience

The main target audience for this report are the Final Year Project evaluation panel, the project supervisor, software developers, researchers, and stakeholders who are interested in retail inventory management and demand forecasting. It has also been written with the reader in mind who would like to learn about the use of data-driven technologies for small and medium sized retailers in order to better plan their inventories and minimize risk from excess inventory and stock out situations.

1.3 Definitions, Acronyms, and Abbreviations

StockSense: The proposed multi-tenant SaaS platform for retail inventory management, demand forecasting, and inventory recommendations.

FYP: Final Year Project.

POS: Point of Sale.

SaaS: Software as a Service.

AI: Artificial Intelligence.

ML: Machine Learning.

ERP: Enterprise Resource Planning.

CSV: Comma-Separated Values.

UI: User Interface.

UX: User Experience.

API: Application Programming Interface.

CRUD: Create, Read, Update, Delete.

MVP: Minimum Viable Product.

SMR: Small and Medium-sized Retailers.

1.4 Conclusion

In conclusion, StockSense is recommended to support small and medium business-store owners in better decision making on inventory management, by combining sales management and actionable sales recommendations and forecasting. The platform is centered around the challenge of stock out and dead stock - stock out at the product level, dead stock at the variant and location level. It also features prioritizing the movement of current stock between branches when necessary and also identifies slow moving stock quickly enough to take preventative measures.

This project merges a free POS layer with subscription-based forecasting and inventory optimization capabilities for businesses to leverage their existing sales data without the need of a complex enterprise system. StockSense aims to bring inventory analytics to smaller retailers by forecasting, redistribution recommendations, markdown optimization, and easy-to-understand explanations of recommendations. The vision and requirements of the project is presented in detail in the following chapters, along with related work, methodology and system design.

Chapter 2 Project Vision

StockSense aims to give the small and medium sized retailer a low cost solution for analyzing their sales pattern and making data-driven decisions on stock. It targets any retailer that stocks physical products with variants, sizes, or multiple stores, such as clothing, footwear, and similar goods, where demand fluctuates greatly by product, size and store location.

2.1 Problem Domain Overview

Retail inventory management is a process that's used to decide what products should be offered, how much, and where. For small businesses these are normally made based on experience and intuition more than formal forecasting. However, StockSense solves this issue by gathering historical sales data and then applying it to forecast demand across product size and location, and then making a recommendation – for instance, reordering stock, moving stock from one branch to another, or even discounting stock that is on the verge of expiring. The system is supposed to be multi tenant software-as-a-service (SaaS) with each business being a standalone tenant that maintains its own products, stores, inventory, sales, forecasts and recommendations. Historical sales reports can be uploaded as a csv or excel file for businesses with an external POS or spreadsheet, or sales can be recorded by businesses with the built-in POS functionality.

2.2 Problem Statement

It can be difficult for retailers to turn past sales information into product, size and location-based inventory decisions. This may result in overstocking, dead stock, unnecessary investment and stock out losses. Hence an integrated solution is required to assist with demand forecasting and inventory optimization, while providing data-driven stock management.

2.3 Problem Elaboration

The main problems addressed by StockSense are:

- Retailers lack the tools to analyze there previous sales data.
- Existing enterprise demand-forecasting solutions can be expensive and technically complex for small businesses.
- Demand can differ between stores, making aggregate product-level forecasts insufficient.

- Demand can also differ significantly between product sizes or variants.
- A store may have excess inventory while another branch needs the same product.
- Slow-moving inventory may only be identified after it has already become dead stock.
- Business owners could find machine-learning-based recommendations difficult to trust if the reasoning behind them is not explained.

2.4 Goals and Objectives

The main objectives of StockSense are:

- Design a multi-tenant platform that allows multiple small businesses to securely manage their sales and inventory data.
- Develop a data ingestion and cleaning pipeline for CSV and Excel sales reports, normalized into a schema of product, size, store/location, date, and quantity sold.
- Develop a demand-forecasting model for product-size-location combinations.
- Develop a recommendation engine that determines what to reorder, how much to reorder, and where existing inventory should be redistributed.
- Develop a markdown/discount optimizer for slow-moving stock.
- Provide explanations and estimated financial impact for every recommendation.
- Evaluate forecast performance using standard forecasting error metrics and estimate potential dead-stock reduction through historical-data simulation.
- Provide the platform under a freemium model consisting of a free POS layer and paid forecasting and recommendation features.

2.5 Project Scope

StockSense target companies have products like clothing, footwear, or any other physical product with variants, sizes, or stores that are out of stock or dead stock. It will allow several companies to be tenants on the platform with their stores, products, inventories and sales history being managed separately. The system will enable multi-tenant business management, store and product/variant management, sales and inventory recording over POS, CSV/Excel historical data upload, data cleaning/normalization, product-size-location demand forecast, inventory redistribution/reordering recommendations, markdown and discount recommendations, explainable recommendations with estimated impact on finances, visual-

ization in dashboard.

First, the forecasting period will be targeted on weekly and monthly demand, instead of a long-term prediction of the trend. For full automatic POS integrations to external systems, it's probably a stretch, but manual/batch upload (CSV/Excel) will be supported with the core data-ingestion mechanism.

2.6 Sustainable Development Goal (SDG)

StockSense aligns with the following United Nations Sustainable Development Goals.

2.6.1 SDG-8 (Decent Work and Economic Growth)

The technology that StockSense provides and enables is something that previously was a privilege of the large retailer and StockSense is unlocking and enabling for small retailers, helping to grow and improve the productivity of small enterprises, which are a primary driver of jobs.

2.6.2 SDG-9 (Industry, Innovation, and Infrastructure)

By providing data-driven innovation, StockSense helps retailers more effectively utilize existing capital, stores space and stock, and minimize excess accumulation of unsold inventory, helping to use resources responsibly.

2.7 Constraints

The main challenges of StockSense include limited availability and quality of historical sales data, the need for new businesses to upload sufficient sales history before reliable forecasts can be generated, and the dependence of forecast accuracy on data quality and quantity. Public datasets may not represent small retailers accurately. The system initially focuses on physical products, such as clothing and footwear, and short- to medium-term demand forecasting. Additionally, redistribution may not always be cost-effective due to transportation and handling costs, transfer delays, and the risk of stockouts at the source branch.

2.8 Business Opportunity

StockSense is a freemium SaaS startup. The basic POS features are included for free, enabling small businesses to keep track of their inventory and sales without paying any subscription fees. A paid subscription provides advanced functionality, such as demand forecasting, inventory redistribution, markdown optimization, benchmarking, and explainable financial recommendations. This model enables

companies to start leveraging the platform with a minimum upfront investment, yet offers the potential for a seamless evolution for stores aiming advanced analytics.

2.9 Stakeholders Description / User Characteristics

The key users of StockSense are business owners, who view business level performance, inventory, forecasts, and recommendations; branch managers, who view inventory level performance, sales, forecasts, and recommendations for their branch; and the system administrator, who manages overall platform level operations, tenants, subscriptions, and system configuration.

Table 2.1: Stakeholders of StockSense

Stakeholder	Responsibilities
Business Owner	Monitor business performance, inventory, forecasts, and recommendations.
Store Manager	Monitor branch-level sales, inventory, forecasts, and stock movement.
System Administrator	Manage tenants, platform configuration, and system-level operations.

2.9.1 Key High-Level Goals and Problems of Stakeholders

Business owners need an easy solution that makes existing sales data into meaningful inventory decisions. Managers need to see what they're selling and the demand for it at the store level. The system administrator needs ways to keep tenants separate and platform running. The common problem is that there's no simple, affordable system which can integrate sales recording, forecasting and inventory suggestions into a single platform.

2.10 Conclusion

This chapter explained the problem domain, goals, scope, and the stakeholders of StockSense. This platform focuses on providing a solution that helps small and medium-sized businesses leverage existing sales data to inform actionable business decisions for inventory management—without the expense or complexity of enterprise solutions. The next chapter examines the literature and current commercial products related to this project.

Chapter 3 Literature Review / Related Work

This chapter summarizes the academic research on problem formulation, demand forecasting, inventory redistribution and markdown optimization, as well as reviews commercial platforms that are relevant to similar problems. This chapter ends with an identification of the gap StockSense is intended to fill and the modeling approach that was chosen for this project.

3.1 Definitions and Acronyms

3.1.1 Definitions

Demand forecasting: The process of estimating future customer demand for a product using historical sales data.

Lateral transshipment: The movement of inventory between locations at the same level of a supply chain, such as between two retail branches, rather than from a warehouse.

Markdown optimization: The process of determining the size and timing of a price discount to sell slow-moving inventory before it becomes unsellable.

SKU: Stock Keeping Unit, a unique identifier for a specific product variant (e.g., a particular size and colour).

3.1.2 Acronyms

MAPE: Mean Absolute Percentage Error.

RMSE: Root Mean Squared Error.

ERP: Enterprise Resource Planning.

POS: Point of Sale.

ML: Machine Learning.

3.2 Detailed Literature Review

Research on fashion retail also considers forecasting demand for new items, where limited sales history makes early prediction especially difficult [1].

3.2.1 Exploring the Use of Deep Neural Networks for Sales Forecasting in Fashion Retail

Summary: Loureiro, Miguéis, and da Silva [2] compare deep neural networks against shallow methods such as support vector regression for forecasting the sales of new fashion products, using a real

dataset from a fashion retailer. They combine data-mining features describing a product's physical and distribution characteristics with expert-defined predictive variables.

Related work on online retail also combines demand forecasting with price optimization [3], which is relevant to StockSense's joint forecasting and markdown objectives.

Critical Analysis: The study not only shows the superiority of deep learning over shallow models for new-product sales forecasting in the fashion retail domain, but it also uses a real company dataset and not a synthetic one. It does make a prediction at the new product level, though, and the data is from a single large retailer; this doesn't make the results of this model easy to apply to a small enterprise with numerous outlets.

Relationship to StockSense: This paper justifies the selection of learning-based demand forecasting over demand estimation done manually and the use of product attributes as predictive features is directly connected to how StockSense will have to forecast demand for a product in a specific store.

3.2.2 Clearance Pricing Optimization for a Fast-Fashion Retailer

Summary: Caro and Gallien [4] describe a forecasting-and-optimization process that replaced Zara's manual markdown decision-making. A field experiment across all Belgian and Irish stores during a fall-winter season showed the new process increased clearance revenue by approximately six percent, and the process was subsequently adopted company-wide.

Critical Analysis: It's an uncommon, documented and now applied – at scale, with a revenue impact – markdown-optimization model. The major drawback of it for StockSense is that it is intended for a very large vertically-integrated retailer, with extensive historical pricing data for thousands of styles, that will not exist in many of the small retailers at which StockSense operates.

Relationship to StockSense: The paper confirms the fundamental concept behind StockSense' markdown optimizer: A formal forecasting/pricing process can be better than a manual and instinctual markdown decision. StockSense takes this concept and applies it to a smaller data set, but with a mindset of "markdown" not as a clearance-event process for large retailers, but as a "preventive, early-warning" measure.

3.2.3 Inventory Models with Lateral Transshipments: A Review

Summary: Paterson, Kiesmüller, Teunter, and Glazebrook [5] review the academic literature on lateral transshipments — stock movements between locations of the same echelon — categorizing research by whether transshipments are proactive (planned in advance) or reactive (triggered by a stockout).

Critical Analysis: As a review paper, it provides broad coverage of transshipment models and clearly identifies gaps in the literature, but it does not propose a specific algorithm suited to a small retailer with only two or three branches, nor does it address how transshipment decisions should be combined with markdown or reorder decisions.

Relationship to StockSense: This review establishes the academic basis for treating inter-branch stock transfer as a distinct decision from reordering or discounting. StockSense builds on the proactive-transshipment framing described here, since its redistribution recommendations are intended to prevent dead stock and stockouts before they occur, rather than reacting to an empty shelf.

3.2.4 Forecasting Across Time Series Databases Using Recurrent Neural Networks on Groups of Similar Series: A Clustering Approach

Summary: Bandara, Bergmeir, and Smyl [6] propose clustering related time series and training a single global recurrent neural network per cluster, rather than one model per series, to improve forecast accuracy when individual series have limited history.

Critical Analysis: The clustering-based global model approach is well suited to situations with many short or sparse series, which is a strong match for a small retailer's sales history. A limitation is that the approach was evaluated primarily on general time-series benchmarks rather than retail-specific size- and store-level sales data.

Relationship to StockSense: This paper supports StockSense's choice of a single shared forecasting model trained across many products and stores, rather than one model per product-size-store combination, since an individual small retailer's sales series are often too short for an accurate standalone model.

3.2.5 M5 Accuracy Competition: Results, Findings, and Conclusions

Summary: Makridakis, Spiliotis, and Assimakopoulos [7] report the results of the M5 forecasting competition, in which thousands of teams forecasted 42,840 hierarchical retail sales series for Walmart. The competition found that machine-learning approaches, particularly gradient-boosted tree models such as LightGBM, consistently outperformed traditional statistical methods.

Critical Analysis: The M5 competition is one of the most rigorous, large-scale empirical comparisons of forecasting methods in retail, and its findings are widely cited as a benchmark for which model families perform well on real retail sales data. Its scale, however, reflects a very large retailer with extensive historical data; the competition does not directly address forecasting for small retailers with short sales histories.

Relationship to StockSense: The competition’s conclusion that gradient-boosted trees outperform simpler statistical baselines on hierarchical retail sales data directly informs the selection of a forecasting model family for StockSense’s product-size-location forecasts.

3.2.6 Demand Forecasting for Improved Inventory Management in Small and Medium-Sized Businesses

Summary: Purnamasari, Permadi, Saepudin, and Agusdin [8] compare machine-learning approaches, including an ensemble gradient-boosting model, against classic statistical projection methods for demand forecasting in small leathercraft businesses, using K-means clustering to group products by similarity before forecasting.

Critical Analysis: This is one of the few studies that directly targets small-business demand forecasting rather than large retail chains, and its finding that machine-learning methods outperform classic statistical approaches is directly relevant. The study is limited to a single small-business dataset in one product category, and does not address size- or location-level demand.

Relationship to StockSense: This paper is the most similar to StockSense in terms of its target audience – a small business with no data-science team. Its finding that gradient-boosting-based methods outperform classic statistical approaches on small-business sales data supports StockSense’s choice of a gradient-boosted model with statistical baselines for comparison.

3.2.7 Retail Forecasting: Research and Practice

Summary: Fildes, Ma, and Kolassa [9] present a comprehensive review of academic research in retail forecasting encompassing strategic, tactical and operational forecasting requirements and explore the disconnect between academic forecasting research and practice in retail stores.

Critical Analysis: It is a survey of the field, and thus can serve as a map of the retail-forecasting research landscape, and clearly shows that new-product forecasting is not studied as much as product forecasting. It is a review and does not itself propose or test a model.

Relationship to StockSense: The paper’s finding that most published research and tools focused on retail forecasting still rely on a data-science team and an IT infrastructure is consistent with the premise of this project: that existing research and tools are not well suited to small retailers, and StockSense aims to address this gap.

3.2.8 Principles and Algorithms for Forecasting Groups of Time Series: Locality and Globality

Summary: Montero-Manso and Hyndman [10] describe conditions under which it is preferable to predict a set of related time series using a shared (global) model rather than a separate (local) model for each series, and demonstrate that global models can outperform local models even when there is no obvious similarity between the series.

Critical Analysis: The paper offers a sound theoretical and empirical foundation for global forecasting models, directly applicable to justify StockSense’s model-selection approach. It is a general time-series paper, not a retail-specific one, so its recommendations need to be adapted to the specific structure of product-size-store sales data.

Relationship to StockSense: This theoretical result supports StockSense’s use of one shared forecasting model across all of a tenant’s products and stores, rather than training a separate model for each product-size-store series, since the latter would require more historical data than a small retailer is likely to have.

3.2.9 Reducing the Cost of Demand Uncertainty Through Accurate Response to Early Sales

Summary: Fisher and Raman [11] show that fashion retailers who shorten lead times and react to early-season sales, rather than committing to production and ordering decisions months in advance, can substantially cut costs. Applied with a major ski-wear firm, their method increased profits by roughly 60% relative to the firm’s existing informal response process.

Critical Analysis: This is a classic, field-validated result demonstrating that acting on early demand signals outperforms manual, instinct-based stocking decisions. Its limitation for StockSense is that it addresses a single-season production and ordering decision for one retailer, rather than ongoing, store-level redistribution between branches.

Relationship to StockSense: This paper is early, influential evidence for the general principle StockSense is built on: that reacting quickly to observed demand, rather than waiting for stock to sit unsold, reduces both lost sales and excess inventory. It supports treating recent sales velocity as a key signal for both the recommendation engine and the markdown optimizer.

3.2.10 Sales Forecasts in Clothing Industry: The Key Success Factor of the Supply Chain Management

Summary: Thomassey [12] reviews forecasting practices in the clothing industry and proposes forecasting models designed to handle the short life cycles, strong seasonality, and sparse historical data typical of apparel sales.

Critical Analysis: The paper directly addresses the data sparsity and seasonality problems that make apparel demand forecasting harder than forecasting for staple goods, which is highly relevant to StockSense's target product categories. It is focused on a single national clothing distributor, so its specific models may not transfer directly to a small multi-branch retailer.

Relationship to StockSense: This paper reinforces the project's recognition that size- and variant-level apparel demand is noisier and more seasonal than product-level demand overall, supporting StockSense's decision to forecast at the product-size-location level rather than at the product level alone.

3.2.11 Predictive Big Data Analytics for Supply Chain Demand Forecasting: Methods, Applications, and Research Opportunities

Summary: Seyedan and Mafakheri [13] survey the methods and applications of big-data-driven demand forecasting across supply chains, classifying approaches and identifying open research gaps.

Critical Analysis: As a broad survey, it usefully maps the landscape of data-driven forecasting techniques and highlights that most existing applications assume large, centrally managed supply chains with dedicated analytics capability. It does not propose or test a specific model itself.

Relationship to StockSense: This survey's identification of a gap between advanced forecasting techniques and organizations lacking analytics infrastructure reinforces the premise of this project: that small retailers are left out of advances that assume a data team is already in place.

3.2.12 Forecasting: Theory and Practice

Summary: Petropoulos et al. [14] provide a wide-ranging review of forecasting theory, methods, and practice, covering model selection, evaluation, and the practical challenges of deploying forecasts in real organizations.

Critical Analysis: The review's breadth makes it a useful reference point for forecast evaluation practices and model-selection principles, though, being an encyclopedic review, it does not focus specifically on retail or small-business contexts.

Relationship to StockSense: The paper's guidance on forecast evaluation and model selection informs

how StockSense will compare its gradient-boosted forecasting model against simpler statistical baselines, as described in the forecast evaluation approach adopted for this project.

3.2.13 From Predictive to Prescriptive Analytics

Summary: Bertsimas and Kallus [15] propose a framework for converting predictive models into prescriptive decisions, demonstrating their approach on a real inventory-management problem for a large media distributor.

Critical Analysis: The paper is a strong theoretical basis for moving beyond a plain forecast to an actionable decision, which is central to how StockSense is meant to work. Its demonstrated case involves a very large-scale distribution operation, which is a different setting from a small multi-branch retailer.

Relationship to StockSense: This paper supports StockSense's core design choice of not stopping at a forecast, but converting it directly into a concrete, actionable recommendation (redistribution, reorder, or markdown), which is the basis of the recommendation engine described in this project.

3.2.14 Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead

Summary: Rudin [16] argues that for high-stakes decisions, inherently interpretable models should be used instead of black-box models paired with separate explanation techniques, since post-hoc explanations of black-box models can be unreliable.

Critical Analysis: The argument is well-supported and widely cited, though it is framed around high-stakes domains such as criminal justice and healthcare rather than retail inventory decisions, where the stakes of an individual recommendation are comparatively low.

Relationship to StockSense: While StockSense's recommendations are lower-stakes than the domains Rudin discusses, the same underlying concern applies: a business owner is unlikely to trust or act on a recommendation they cannot understand. This supports StockSense's design choice to generate a plain-language explanation alongside every recommendation, rather than presenting an unexplained output.

3.2.15 Desperately Seeking Shelf Availability: An Examination of the Extent, the Causes, and the Efforts to Address Retail Out-of-Stocks

Summary: Corsten and Gruen [17] report on a worldwide study of retail out-of-stocks, finding that stockout rates had not improved over the preceding decade despite industry efforts, and that a meaningful share of customers switch stores entirely rather than switch products when faced with an out-of-stock.

Critical Analysis: This is a large-scale, widely cited study that establishes stockouts as a persistent, costly, and customer-facing problem, grounding the motivation for this project in real consumer behavior rather than only internal cost. The study focuses on fast-moving consumer goods, not apparel or footwear specifically.

Relationship to StockSense: This paper strengthens the business case for prioritizing redistribution and timely reordering: since customers facing a stockout may leave the store entirely rather than buy a different item, preventing a stockout has a larger business impact than simple lost-sale accounting would suggest.

3.3 Literature Review Summary Table

Table 3.1: Summary of Literature Review

No.	Paper Title	Year	Summary	Relevance to StockSense
1	Exploring the Use of Deep Neural Networks for Sales Forecasting in Fashion Retail	2018	Deep learning outperforms shallow models for new fashion-product sales forecasting	Supports learning-based forecasting of fashion product sales
2	Clearance Pricing Optimization for a Fast-Fashion Retailer	2012	Formal markdown model increased clearance revenue at Zara by 6%	Validates preventive markdown optimization as a business lever
3	Inventory Models with Lateral Transshipments: A Review	2011	Reviews proactive and reactive stock-transfer models between locations	Basis for treating branch-to-branch redistribution as a first-class action
4	Forecasting Across Time Series Databases Using RNNs on Groups of Similar Series	2020	Clustering similar series and forecasting them with one shared model improves accuracy	Basis for shared/clustered forecasting models
5	M5 Accuracy Competition: Results, Findings, and Conclusions	2022	Gradient-boosted tree models outperformed statistical baselines on large-scale retail data	Informs choice of forecasting model family
6	Demand Forecasting for Improved Inventory Management in SMBs	2023	ML outperforms classic methods for small-business demand forecasting	Directly matches StockSense’s target user segment
7	Retail Forecasting: Research and Practice	2022	Survey of retail-forecasting research and the gap with practitioner adoption	Confirms the small-retailer research and tooling gap

Continued on next page

No.	Paper Title	Year	Summary	Relevance to StockSense
8	Principles and Algorithms for Forecasting Groups of Time Series	2021	Formalizes when a shared global model outperforms per-series local models	Theoretical basis for shared forecasting models
9	Reducing the Cost of Demand Uncertainty Through Accurate Response to Early Sales	1996	Quick Response to early sales data increased a fashion retailer's profits by ~60%	Validates acting on recent sales velocity for stocking decisions
10	Sales Forecasts in Clothing Industry: The Key Success Factor of Supply Chain Management	2010	Reviews forecasting models suited to apparel's short life cycles and seasonality	Supports forecasting at the product-size-location level
11	Predictive Big Data Analytics for Supply Chain Demand Forecasting	2020	Survey of data-driven forecasting methods and research gaps across supply chains	Confirms the gap for organizations without analytics infrastructure
12	Forecasting: Theory and Practice	2022	Encyclopedic review of forecasting theory, methods, and evaluation practice	Informs forecast evaluation and model-selection approach
13	From Predictive to Prescriptive Analytics	2020	Framework for converting predictive models into actionable decisions	Basis for converting forecasts into recommendations, not just numbers
14	Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead	2019	Argues interpretable models are more trustworthy than explained black boxes	Supports explainable, plain-language recommendations
15	Desperately Seeking Shelf Availability	2003	Worldwide study finds stockouts persistent; many customers switch stores entirely	Strengthens business case for prioritizing stockout prevention

3.4 Related Work

SAP IBP is an enterprise demand-sensing and planning platform designed to take in real-time signals from the point of sale to fine-tune forecasts and prevent stockouts and overstocking. It's designed to work for large businesses with a planning staff already in place and enterprise licensing costs, and cannot be used by a small retailer running two or three stores, and who has no budget for enterprise licensing.

3.4.1 Blue Yonder

Home to large retailers like Sainsbury's, Blue Yonder, a part of Panasonic, offers machine learning-based demand planning and retail-planning software. It has causal forecasting (not only based on historical averages) and also offers the ability to explain its predictions in a "glassbox" manner. It's on an enterprise level like SAP, so it takes a lot of effort to implement and technical infrastructure, which a small business does not have.

3.4.2 Nextail

Nextail is an AI-powered merchandising platform specifically designed for fashion, footwear and apparel retailers that enables in-season allocation, replenishment and inventory rebalancing between stores that helps to ensure full-price sell-through and minimise end-of-season markdowns. It's the most similar thing to StockSense's redistribution focus, but it applies to mid-size and large fashion chains, costs a lot more, and gives its recommendations as a part of a larger merchandising tool as opposed to a free one for a very small retailer.

3.4.3 Shopify POS (with Stocky)

In addition to multi-location inventory management, stock transfer between locations, low-stock alerts, shopify POS has inventory-rebalancing recommendations between stores and demand-forecasting reorder suggestions through a module called Stocky and Sidekick AI assistant. It's the easiest of the tools mentioned for a small retailer who is selling on Shopify. The core level subscription fee, however, does not include the advanced inventory features, which are available only with paid Shopify plans, while its forecasting and rebalancing capabilities are at product level—not size or variant level.

3.4.4 Lightspeed Retail

Offering stock transfer, custom reorder points per location and reports that highlight slow moving items, Lightspeed Retail is a POS system designed for multi-location retailers with a large number of SKUs. It also lacks variant-level or size-level demand prediction, explainable recommendations or an independent markdown-timing tool as Shopify has, and doesn't come with a free tier—meaning that retailers will have to pay money for the POS, even if they don't need advanced inventory analytics.

3.4.5 Odoo

Odoo is an open-source business platform whose free Community edition includes a Point of Sale app and an Inventory app with built-in reordering rules and multi-warehouse stock transfers. It is the closest existing tool to StockSense's free-POS model, since the core POS and inventory tracking cost nothing. However, Odoo's free edition has no built-in demand forecasting, size/variant-level analysis, or explainable recommendations; such capabilities exist only as separately priced third-party marketplace apps or within its paid Enterprise plans, and even those add-ons generally forecast at the product level rather than prioritizing branch-to-branch redistribution the way StockSense does.

3.5 Related Work Summary Table

Table 3.2: Summary of Related Work

No.	Platform	Target Users	Gap Relative to StockSense
1	SAP Integrated Business Planning	Large enterprises with ERP	Requires ERP infrastructure and enterprise budget; no free tier
2	Blue Yonder	Large retail chains	Enterprise-only; requires dedicated planning/IT teams
3	Nextail	Mid-size to large fashion chains	Closest in focus to redistribution, but priced and scaled for many-store chains, not a free small-retailer entry point
4	Shopify POS (Stocky/Sidekick)	Shopify merchants of any size	Product-level, not size/variant-level, forecasting; advanced features gated behind paid plans
5	Lightspeed Retail	Multi-location, high-SKU retailers	No demand forecasting or explainable recommendations; no free tier
6	Odoo (Community)	Small businesses wanting free, self-hosted POS/Inventory	Free tier has no demand forecasting or explainable recommendations; forecasting only via paid addons, product-level not size/variant-level

3.6 Conclusion

The literature demonstrates that machine-learning predictions and formal markdown and redistribution methods can enhance inventory choices and income. There are solutions out there, though, that are geared toward larger retailers, who have infrastructure in place and data teams. Multiple solutions were reviewed, but few are free to use and include explainable, size- and location-level forecasting with redistribution as a priority for small, multi-branch retailers. StockSense is designed to fill this void.

3.7 Selected Model

According to this review, StockSense’s forecasting algorithm will employ the gradient-boosted tree model family as the main one, as this family is the one that was found to be performing well in the M5 competition on the hierarchical retail sales data, while simple statistical methods are kept as a reference to compare the performance of the other models. The recommendations for redistribution will be based on the proactive-transshipment framing from Paterson et al., with the emphasis put on moving stocks between branches rather than ordering and/or discounting. The markdown optimizer will do what Caro and Gallien did - it will mark items down even before they are considered unsellable.

Chapter 4 Software Requirement Specifications

This chapter describes the functional and non-functional requirements of StockSense, derived from the project goals and scope described in Chapter 2.

4.1 List of Features

- Multi-tenant business registration and secure login.
- Free built-in point-of-sale (POS) for recording sales and tracking inventory.
- CSV/Excel upload of historical sales reports for businesses using an external POS.
- Data validation, cleaning, and normalization of uploaded sales data.
- Product, size/variant, and store/location management.
- Product-size-location demand forecasting.
- Inventory redistribution recommendations between branches.
- Reordering recommendations.
- Markdown/discount optimizer for slow-moving stock.
- Explainable recommendations with estimated financial impact.
- Web dashboard for sales, inventory, forecasts, and recommendations.
- Freemium subscription management (free POS tier, paid analytics tier).
- Cross-tenant benchmarking (where data permits).

4.2 Functional Requirements

4.2.1 Functional Requirements of the Business Owner

- FR-1: The business owner shall be able to register the business as a tenant and create an account.
- FR-2: The business owner shall be able to add, edit, and remove stores belonging to the business.
- FR-3: The business owner shall be able to view consolidated sales, inventory, and forecast data across all stores.
- FR-4: The business owner shall be able to view and act on redistribution, reorder, and markdown recommendations.
- FR-5: The business owner shall be able to upgrade or downgrade the subscription tier.
- FR-6: The business owner shall be able to invite and manage store-manager accounts.

4.2.2 Functional Requirements of the Store Manager

- FR-7: The store manager shall be able to record sales through the POS interface.
- FR-8: The store manager shall be able to view current inventory for their assigned store.
- FR-9: The store manager shall be able to view demand forecasts and recommendations relevant to their store.
- FR-10: The store manager shall be able to confirm or reject a redistribution recommendation involving their store.

4.2.3 Functional Requirements of the System

- FR-11: The system shall separate tenant's data from other tenants.
- FR-12: The system should be able to accept and validate sales-report uploads in CSV and Excel.
- FR-13: The system shall normalize uploaded data into a common schema (product, size, store, date, quantity).
- FR-14: After each sale recorded, the system shall automatically update the inventory.
- FR-15: The system shall generate demand forecasts at the product-size-location level.
- FR-16: The system shall compare forecasted demand against current inventory to identify shortages and surpluses.
- FR-17: When more than one branch has surplus stock and another branch has a forecasted shortage of the same product/size, the system shall provide a redistribution recommendation.

- FR-18: The system shall generate a reorder recommendation when no suitable surplus exists elsewhere.
- FR-19: The system shall identify inventory at risk of becoming dead stock and generate a markdown recommendation before it becomes unsellable.
- FR-20: Recommendations shall be accompanied by a clear statement of its consequences and a reasonable cost estimate.
- FR-21: The system shall restrict access to paid features based on the tenant's subscription tier.
- FR-22: The system shall evaluate forecast accuracy using standard error metrics (e.g., MAPE, RMSE).

4.3 Quality Attributes

4.3.1 Reliability

The system shall provide consistent forecasts and recommendations from the same input data, and shall not crash or corrupt tenant data on the server on receiving malformed uploads.

4.3.2 Maintainability

The forecasting, recommendation, markdown and pos logic shall be built as separate modules enabling one to be changed, retrained or replaced without affecting other modules.

4.3.3 Usability

The dashboard should share forecasts and suggestions in a business owner-friendly format, accessible without any data-science or machine-learning expertise.

4.3.4 Correctness

The quantity of the inventory items shown to the user will always be the latest amount sold and transferred based on recommendations.

4.3.5 Efficiency

The system shall be able to provide a forecast and recommendations for the entire product catalog of a tenant within a reasonable time, depending on the size of the uploaded data set.

4.3.6 Flexibility

The system shall be able to add new forecasting models or recommendation rules without redesigning the data ingestion and/or dashboard layers.

4.4 Non-Functional Requirements

4.4.1 Performance

It should handle a typical small retailer's monthly sales upload (a few thousand rows) and be able to provide forecasts in a couple of minutes.

4.4.2 Reliability

The system should be highly available for both the POS and the dashboard, as the store manager might rely on the POS during the business hours.

4.4.3 Security

Tenant information will be isolated at the database level. Secure hashing algorithm shall be used to store passwords. Communication between client and server shall be encrypted (HTTPS/TLS).

4.4.4 Compatibility

Access to the dashboard should be via modern web browser, with no need to install a native application.

4.5 Assumptions

- Businesses have, or are willing to generate, sales data at the product-size-store level.
- Users have basic computer literacy and a stable internet connection.
- Uploaded CSV/Excel files follow, or can be mapped to, a reasonably consistent column structure.
- A single store belongs to exactly one tenant at a time.
- Forecasting accuracy will improve as more historical data accumulates for a given tenant.

4.6 Use Cases

Table 4.1: UC-01: Register Business

Name	Register Business
Actors	Business Owner
Summary	A new business signs up as a tenant on StockSense.
Pre-Conditions	The business is not already registered.
Post-Conditions	A new tenant account is created on the free POS tier.
Basic Flow	1. The owner opens the registration page. 2. The owner enters business and account details. 3. The system creates the tenant and sends a verification email.
Alternative Flow	If the email is already registered, the system displays an error and asks for a different email.

Table 4.2: UC-02: Login

Name	Login
Actors	Business Owner, Store Manager, System Administrator
Summary	A registered user logs in using an email and password.
Pre-Conditions	The user has an account and is not already logged in.
Post-Conditions	A session is established and the user is redirected to the dashboard.
Basic Flow	1. The user opens the login page. 2. The user enters the email and password. 3. The system verifies the credentials and opens the dashboard.
Alternative Flow	If the credentials are invalid, the system displays “Incorrect email or password” and stays on the login page.

Table 4.3: UC-03: Manage Stores

Name	Manage Stores
Actors	Business Owner
Summary	The owner adds, edits, or removes the stores (branches) of the business.
Pre-Conditions	The owner is logged in.
Post-Conditions	The list of stores of the tenant is updated.
Basic Flow	1. The owner opens the Stores page. 2. The owner chooses to add, edit, or remove a store and enters the details. 3. The system validates and saves the change.
Alternative Flow	If a store being removed still has inventory or sales history, the system archives it instead of deleting it.

Table 4.4: UC-04: Manage Store Managers

Name	Manage Store Managers
Actors	Business Owner
Summary	The owner invites store managers, assigns them to a store, or removes their access.
Pre-Conditions	The owner is logged in and at least one store exists.
Post-Conditions	The store manager accounts of the tenant are updated.
Basic Flow	1. The owner opens the Users page. 2. The owner invites a manager by email and selects the store to assign. 3. The system creates the account and sends an invitation.
Alternative Flow	If the email already belongs to another account, the system rejects the invitation and shows an error.

Table 4.5: UC-05: Manage Products and Variants

Name	Manage Products and Variants
Actors	Business Owner
Summary	The owner adds, edits, or removes products and their sizes/variants.
Pre-Conditions	The owner is logged in.
Post-Conditions	The product catalog of the tenant is updated.
Basic Flow	1. The owner opens the Products page. 2. The owner creates or edits a product and its sizes/variants. 3. The system validates and saves the product catalog.
Alternative Flow	If a variant with the same SKU already exists, the system rejects the entry and asks for a different SKU.

Table 4.6: UC-06: Adjust Inventory

Name	Adjust Inventory
Actors	Business Owner, Store Manager
Summary	A user corrects the stock quantity of a variant at a store, for example after a physical count.
Pre-Conditions	The user is logged in and the variant exists.
Post-Conditions	The stock quantity is updated and the adjustment is logged.
Basic Flow	1. The user selects a store and a variant. 2. The user enters the new quantity and a reason. 3. The system updates the inventory and records the adjustment.
Alternative Flow	If the entered quantity is negative, the system rejects the adjustment.

Table 4.7: UC-07: Record Sale (POS)

Name	Record Sale (POS)
Actors	Store Manager
Summary	A store manager records a sale through the built-in POS.
Pre-Conditions	The manager is logged in; the product and size exist in inventory.
Post-Conditions	A sales record is created and inventory is decremented.
Basic Flow	1. The manager selects the product and size sold. 2. The manager confirms the quantity and completes the sale. 3. The system logs the sale and updates inventory.
Alternative Flow	If inventory is insufficient, the system warns the manager before confirming.

Table 4.8: UC-08: Upload Historical Sales Data

Name	Upload Historical Sales Data
Actors	Business Owner
Summary	The owner uploads a CSV or Excel file of historical sales from an external POS or spreadsheet.
Pre-Conditions	The owner is logged in and belongs to a registered tenant.
Post-Conditions	Valid rows are cleaned, normalized, and stored; invalid rows are listed in an upload report.
Basic Flow	1. The owner selects a CSV/Excel file. 2. The system shows the detected columns and the owner maps them to the required fields. 3. The system validates and normalizes the data and shows an upload report. 4. The owner approves the report and the system imports the rows.
Alternative Flow	If the file format is unsupported or unreadable, the system rejects the upload and explains why. Rejected rows are listed in the report with reasons.

Table 4.9: UC-09: View Dashboard and Alerts

Name	View Dashboard and Alerts
Actors	Business Owner, Store Manager
Summary	The user views a summary of sales, inventory, forecasts, recommendations, and alerts.
Pre-Conditions	The user is logged in.
Post-Conditions	The dashboard displays current, tenant-scoped data.
Basic Flow	1. The user logs in and is directed to the dashboard. 2. The system displays sales trends, current inventory, open recommendations, and alerts.
Alternative Flow	If the tenant has no sales data yet, the dashboard asks the user to record or upload sales.

Table 4.10: UC-10: View Demand Forecast

Name	View Demand Forecast
Actors	Business Owner, Store Manager
Summary	The user views the forecasted weekly or monthly demand of a product-size-store combination.
Pre-Conditions	The user is logged in and the tenant has a paid subscription.
Post-Conditions	The forecast is displayed next to the past sales.
Basic Flow	1. The user opens the Forecasts page. 2. The user selects a product, size, and store. 3. The system displays the stored forecast with the past sales.
Alternative Flow	If the tenant has not yet provided enough sales history to train a model, no forecast is shown and the user is asked to record or upload more data. On the free tier, the system asks the user to upgrade.

Table 4.11: UC-11: View and Act on Redistribution Recommendation

Name	View and Act on Redistribution Recommendation
Actors	Business Owner, Store Manager
Summary	The user reviews a suggestion to move stock from one branch to another and approves or dismisses it.
Pre-Conditions	A forecasted shortage at one store coincides with a forecasted surplus of the same item at another.
Post-Conditions	An approved recommendation creates a stock transfer; a dismissed one is closed.
Basic Flow	1. The user opens the Recommendations page and selects a redistribution recommendation. 2. The system displays source and destination stores, quantity, explanation, and estimated financial impact. 3. The user approves it and the system creates the stock transfer and notifies both stores.
Alternative Flow	The user dismisses the recommendation and the system marks it as dismissed.

Table 4.12: UC-12: View Markdown Recommendation

Name	View Markdown Recommendation
Actors	Business Owner, Store Manager
Summary	The user views a suggestion to discount a slow-moving item before it becomes dead stock.
Pre-Conditions	The markdown optimizer has flagged an item as at risk of becoming dead stock.
Post-Conditions	The user sees the recommended discount, explanation, and estimated impact.
Basic Flow	1. The user opens the Recommendations page. 2. The user selects a markdown recommendation. 3. The system displays the item, suggested discount, explanation, and estimated financial impact.
Alternative Flow	The user can dismiss the recommendation if a markdown is not desired.

Table 4.13: UC-13: View Reorder Recommendation

Name	View Reorder Recommendation
Actors	Business Owner
Summary	The user views a suggestion to reorder an item when no surplus exists at other branches.
Pre-Conditions	A forecasted shortage remains after redistribution has been considered.
Post-Conditions	The user sees the suggested reorder quantity with its explanation.
Basic Flow	1. The user opens the Recommendations page. 2. The user selects a reorder recommendation. 3. The system displays the item, store, suggested quantity, and explanation.
Alternative Flow	The user marks the recommendation as actioned or dismisses it.

Table 4.14: UC-14: Manage Subscription

Name	Manage Subscription
Actors	Business Owner
Summary	The owner views the current plan and upgrades or downgrades between the free and paid tiers.
Pre-Conditions	The owner is logged in.
Post-Conditions	The subscription tier is updated and paid features are enabled or disabled accordingly.
Basic Flow	1. The owner opens the Subscription page. 2. The owner selects a plan and confirms the change. 3. The system updates the tier and the available features.
Alternative Flow	If the payment fails, the system keeps the current tier and informs the owner.

Table 4.15: UC-15: Manage Tenants

Name	Manage Tenants
Actors	System Administrator
Summary	The administrator views tenants and manages their status and plans.
Pre-Conditions	The administrator is logged in.
Post-Conditions	The status or plan of the selected tenant is updated.
Basic Flow	1. The administrator opens the Tenants page. 2. The administrator selects a tenant and chooses an action (suspend, reactivate, or change plan). 3. The system applies the change and records it.
Alternative Flow	Users of a suspended tenant cannot log in until the tenant is reactivated.

4.7 Hardware and Software Requirements

4.7.1 Hardware Requirements

- **Server:** Minimum 8 GB RAM, quad-core processor, cloud-hosted or on-premise.
- **Client:** Any modern computer or smartphone with a web browser and internet access.

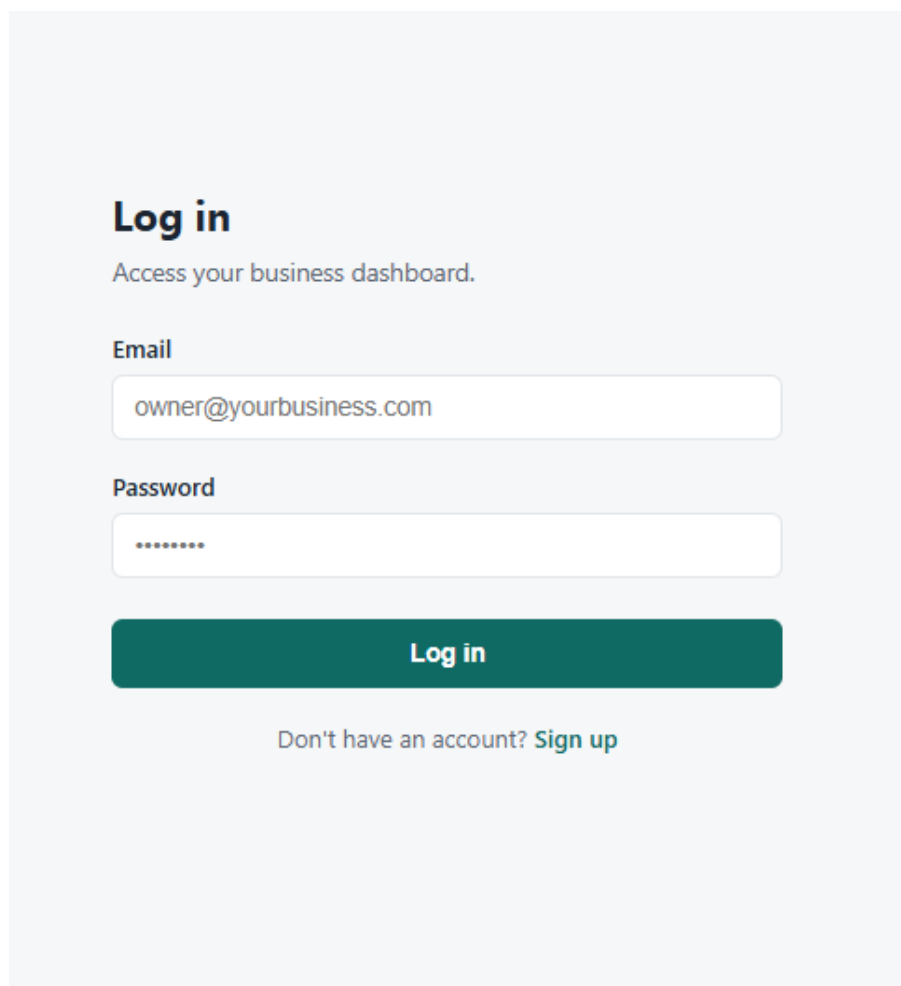
4.7.2 Software Requirements

- **Backend:** Python (FastAPI) for the application and forecasting services.
- **Frontend:** React, for the web dashboard.
- **Database:** PostgreSQL.
- **Forecasting/ML:** scikit-learn and LightGBM (gradient-boosted trees), following the M5-informed model choice from Chapter 3.
- **Operating System:** Linux server environment.

4.8 Graphical User Interface

GUI mockups of each screen, with a description of the users and functionality, are given below.

4.8.1 Login Page



Log in

Access your business dashboard.

Email

owner@yourbusiness.com

Password

.....

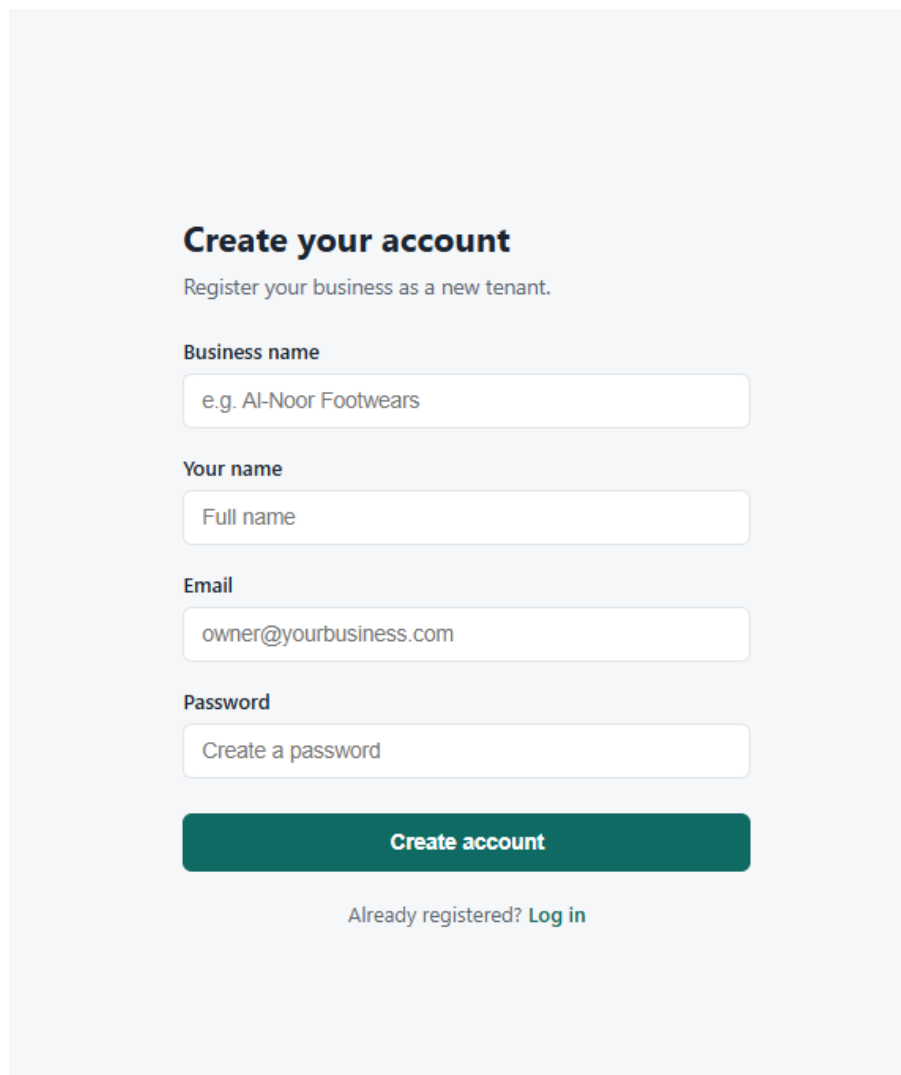
Log in

Don't have an account? [Sign up](#)

Figure 4.1: Login Page

Any registered user, whether a business owner or a store manager, logs in from this page using their email and password.

4.8.2 Signup Page



The image shows a web form for creating a new account. It has a light gray background. At the top, the heading 'Create your account' is in bold, followed by the subtext 'Register your business as a new tenant.' Below this are four input fields: 'Business name' with the placeholder 'e.g. Al-Noor Footwears', 'Your name' with the placeholder 'Full name', 'Email' with the placeholder 'owner@yourbusiness.com', and 'Password' with the placeholder 'Create a password'. A dark green button labeled 'Create account' is positioned below the password field. At the bottom, there is a link that says 'Already registered? Log in'.

Create your account
Register your business as a new tenant.

Business name
e.g. Al-Noor Footwears

Your name
Full name

Email
owner@yourbusiness.com

Password
Create a password

Create account

Already registered? [Log in](#)

Figure 4.2: Signup Page

A new business owner registers their business as a tenant from this page, creating the initial account for StockSense.

4.8.3 Dashboard

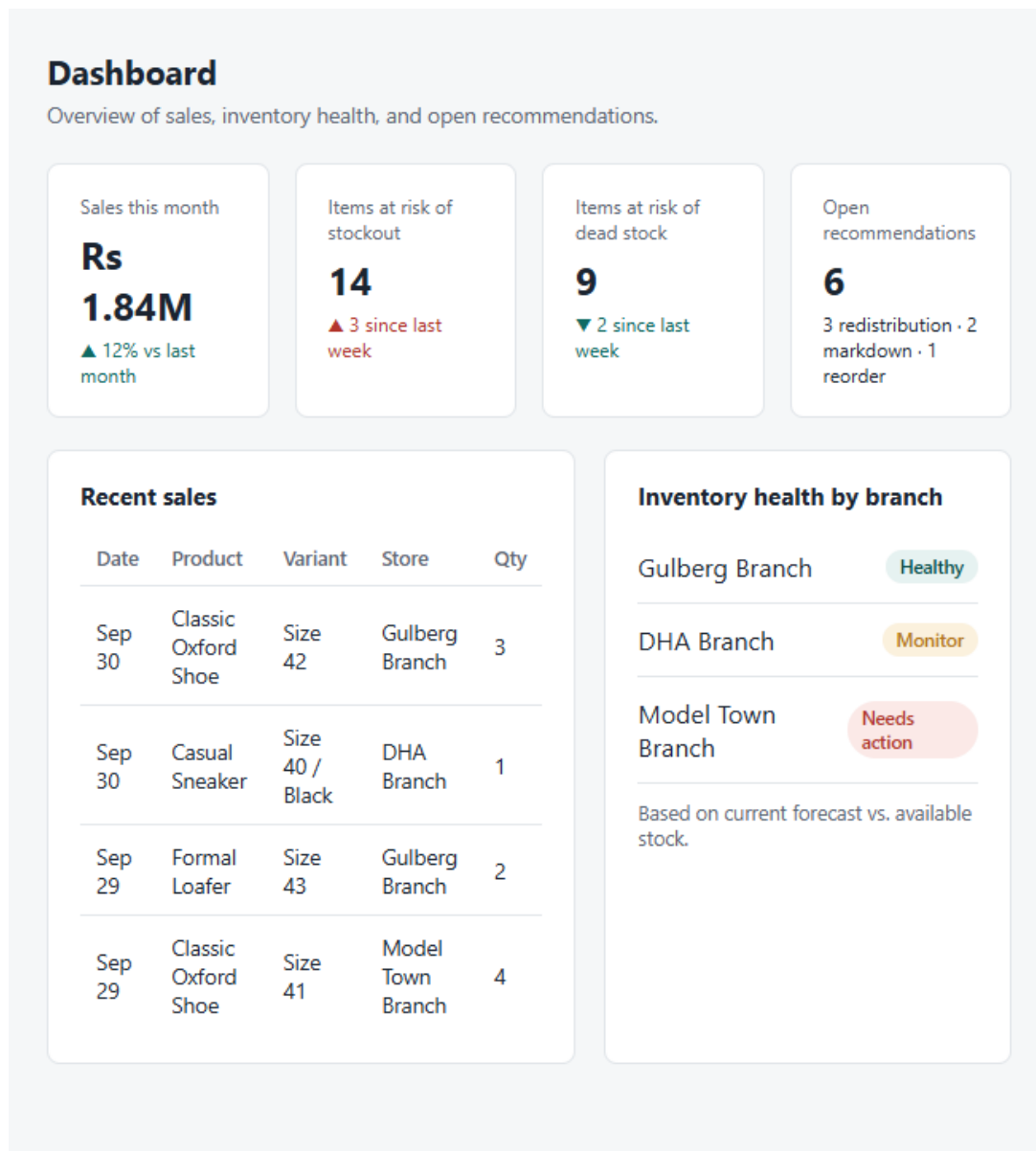


Figure 4.3: Dashboard

On this page, the user can view an overview of monthly sales, items at risk of stockout or dead stock, recent sales activity, and inventory health across branches.

4.8.4 Point of Sale

Point of Sale

Record a sale and keep branch inventory up to date.

Record a sale

Store / branch

Gulberg Branch

Product

Classic Oxford Shoe

Size / variant

Size 40

Quantity

1

Complete sale

Current stock for this item

Gulberg Branch	18 in stock
DHA Branch	2 in stock
Model Town Branch	0 in stock

Stock updates automatically after this sale is recorded.

Figure 4.4: Point of Sale

A store manager records a sale from this page by selecting the branch, product, and variant sold. Inventory is updated automatically once the sale is confirmed.

4.8.5 Upload Sales Report

Upload Sales Report

Import historical sales data from an external POS or spreadsheet.

Upload historical sales report

Already using another POS or a spreadsheet? Upload your sales history as a CSV or Excel file and StockSense will clean and organize it automatically.

Drag and drop your file here

or click to browse · supports .csv, .xlsx

Upload fileDownload sample template

Recent uploads

File	Rows imported	Rows flagged	Status
august_sales.xlsx	1,204	3	Processed
july_sales.csv	1,088	0	Processed

Figure 4.5: Upload Sales Report

Businesses already using an external POS or spreadsheet can upload their historical sales data as a CSV or Excel file from this page, which is then validated and cleaned by the system.

4.8.6 Inventory

Inventory

Current stock levels across all branches.

All stores Gulberg Branch DHA Branch Model Town Branch

Product	Variant	Store	In stock	Status
Classic Oxford Shoe	Size 41	Gulberg Branch	22	Healthy
Classic Oxford Shoe	Size 41	Model Town Branch	0	Out of stock
Casual Sneaker	Size 40 / Black	DHA Branch	31	Slow-moving
Formal Loafer	Size 43	Gulberg Branch	6	Healthy
Formal Loafer	Size 44	DHA Branch	27	Slow-moving

Figure 4.6: Inventory

This page displays current stock levels for every product variant across all branches, filterable by store, along with a status indicator for each item.

4.8.7 Recommendations

Recommendations

Explainable, prioritized actions generated from your sales data.

All (6)Redistribution (3)Markdown (2)Reorder (1)

REDISTRIBUTION**Est. saving: Rs 18,400**

Move 12 units of Classic Oxford Shoe, Size 41 from Gulberg Branch to Model Town Branch

Model Town Branch is forecast to sell out of this size within 9 days, while Gulberg Branch is holding 22 units with slowing demand. Moving stock now avoids both a stockout and a markdown.

Approve transferDismiss

MARKDOWN**Est. recovery: Rs 9,600**

Apply a 15% discount to Casual Sneaker, Size 40 / Black at DHA Branch

Sales velocity for this variant has dropped 68% over the last 4 weeks with no comparable demand at other branches. A discount now is estimated to recover more value than waiting.

Apply discountDismiss

REORDER**Est. demand: 40 units / 2 weeks**

Reorder Formal Loafer, Size 43 for Gulberg Branch

Forecasted demand exceeds available stock at this branch, and no surplus exists at other branches to redistribute from.

Create purchase orderDismiss

Figure 4.7: Recommendations

This page lists the system’s redistribution, markdown, and reorder recommendations, each with a plain-language explanation and an estimated financial impact, which the user can approve or dismiss.

4.9 Database Design

4.9.1 ER Diagram

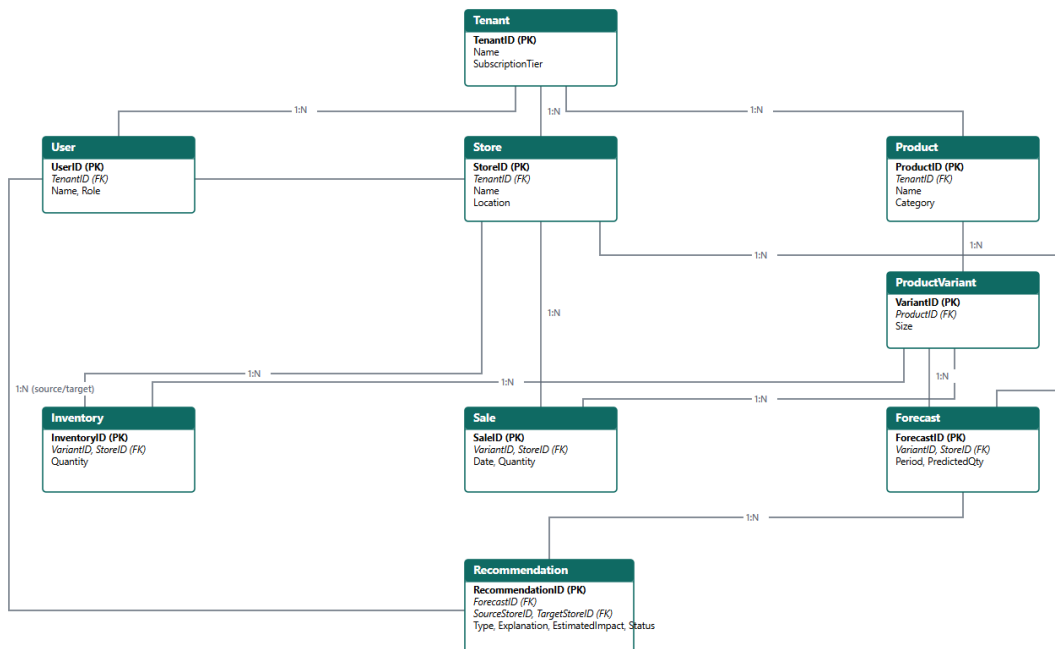


Figure 4.8: ER Diagram

This is the design of the StockSense database. Each Tenant maintains separate Users, Stores, and Products, so that all business data is kept separate. A Product is associated with a number of Variants (e.g. sizes) and Inventory keeps track of how many of each Variant a Store has. Sales records and Forecasts are created at the Variant level per Store; Recommendations (redistribution, markdown, and reorder) are made based upon Forecasts and refer to the Store(s) involved.

The major entities and their relationships are:

- A **Tenant** has many **Users**, **Stores**, and **Products**.
- A **Product** has many **Product Variants** (size/colour combinations).
- A **Store** maintains **Inventory** records per Product Variant.
- A **Sale** references a Product Variant, a Store, and a date/quantity.
- Historical **Sales** feed the **Forecast** entity, generated per Product Variant per Store.
- A **Forecast** is consumed by the **Recommendation** entity (redistribution, reorder, or markdown type).

4.9.2 Data Dictionary

Table 4.16: Data Dictionary for Tenant Table

Field	Type	Description
TenantID	Integer	Unique identifier for each business
Name	String	Business name
SubscriptionTier	String	Free or Paid
CreatedDate	Date	Date the tenant registered

Table 4.17: Data Dictionary for Store Table

Field	Type	Description
StoreID	Integer	Unique identifier for each store
TenantID	Integer	Owning tenant
Name	String	Store/branch name
Location	String	Store address or region

Table 4.18: Data Dictionary for Product Variant Table

Field	Type	Description
VariantID	Integer	Unique identifier for a product-size combination
ProductID	Integer	Parent product
Size	String	Size or variant label
Category	String	Product category

Table 4.19: Data Dictionary for Sale Table

Field	Type	Description
SaleID	Integer	Unique identifier for each sale
VariantID	Integer	Product variant sold
StoreID	Integer	Store where the sale occurred
Date	Date	Date of sale
Quantity	Integer	Units sold

Table 4.20: Data Dictionary for Recommendation Table

Field	Type	Description
RecommendationID	Integer	Unique identifier for each recommendation
Type	String	Redistribution, Reorder, or Markdown
VariantID	Integer	Product variant involved
SourceStoreID	Integer	Store with surplus (redistribution only)
TargetStoreID	Integer	Store with forecasted need
Explanation	String	Plain-language reasoning for the recommendation
EstimatedImpact	Float	Estimated financial impact
Status	String	Pending, Actioned, or Dismissed

4.10 Risk Analysis

- **Technical Risk:** Forecast accuracy may be poor for tenants with very limited historical sales data.
- **Data-Quality Risk:** Inconsistent or incomplete uploaded sales reports may require significant cleaning effort.
- **Adoption Risk:** Small-business owners may be reluctant to trust automated recommendations without a visible track record.
- **Scalability Risk:** As more tenants and historical data accumulate, forecasting jobs must remain performant.
- **Business Risk:** Changing project requirements or limited availability of real business data for testing.

4.11 Conclusion

In this chapter, you learned the functionality and non-functional requirements, use cases, and database design of StockSense. Requirements focus on three pillars identified in the Project Vision and Literature Review: affordable multi-tenant data collection using a free POS, product-size-location demand forecasting based on the literature for model selection, and explainable recommendations that focus on redistribution. The next chapter describes the proposed approach and methodology used to implement these requirements.

Chapter 5 Proposed Approach and Methodology

The proposed approach and methodology is presented in this chapter. The chapter explains how StockSense transforms raw sales data into inventory recommendations. It traces the data from ingestion to validation, construction of the demand series, forecasting, recommendation, optimization of markdowns, explanation, and evaluation. The literature reviewed in Chapter 3 is the basis for each design decision.

5.1 Overall Pipeline

StockSense is a sequential pipeline in which each stage reads data from and writes data to the PostgreSQL database through the modules described in Chapter 6. Sales are entered through the built-in POS or by file upload and are stored in one canonical schema. A demand series is created for each product-size-store combination, features are created from the history of the demand series, and the model provides weekly and monthly demand forecasts. These forecasts are then compared with the current inventory to generate redistribution, markdown, and reorder recommendations, all of which include an explanation and an estimated financial impact.

Forecasting and recommendation run as scheduled background jobs, and their results are stored in the Forecast and Recommendation tables. The model name (`ModelName`) is saved with each forecast, which allows models to be compared or replaced without altering the remainder of the system. To respect tenant isolation, models are trained per tenant: a model is shared across all products, sizes, and stores of one business, but never across businesses.

5.2 Data Ingestion

Sales come from two sources: `Sale` and `SaleItem` records from the built-in POS (`Source = POS`), and CSV or Excel files from an external system (`Source = Upload`). Both sources use the same schema, so the later stages are independent of the source.

CSV files are read with pandas in chunks, while Excel files are read with pandas and `openpyxl` using bounded-sheet or batch processing, to keep memory use manageable for files of up to 100,000 rows. The system displays the detected columns, and the owner maps them to the canonical fields (product, size, store, date, quantity). Validation and import run as a background job. Validated rows are held until the owner approves the upload report, and are then committed in a single transaction. If the import fails, the transaction is rolled back and the data remains as it was before the import.

5.3 Data Validation, Cleaning, and Normalization

The following checks are performed for every row:

- **Structural:** all required fields are present and have the expected data type.
- **Value:** the date is valid, the quantity is numeric, and the price is non-negative. Dates such as 03/04/2025 are ambiguous, so the owner selects the date format when the columns are mapped.
- **Referential:** product, size, and store names are normalized (trimmed, case folded, and sizes converted to a standard form such as “Medium” to “M”) and must match a variant and a store within the same tenant. Unmatched rows are rejected with a reason, and close matches can be suggested using string similarity.
- **Duplicates:** rows repeated across overlapping exports are detected using a composite key (store, variant, date, quantity, price). They are flagged and not deleted, because identical sales can be genuine.
- **Returns and outliers:** negative quantities are treated as returns and reported separately. Extreme quantities are flagged and capped only in the training copy, so the raw record is preserved.

Accepted rows are stored in the canonical format. Rejected and flagged rows are listed in the upload report with reasons, allowing the owner to correct the source file.

5.4 Demand Series Construction

Cleaned sales data are aggregated to the product-size-store level on a weekly and monthly basis. Weekly forecasts are produced from weekly series; monthly forecasts are produced from monthly series so that the forecast horizon and evaluation period use the same time granularity:

$$y(v, s, t) = \sum_{i \in \mathcal{S}(v, s, t)} q_i, \quad (5.1)$$

where $\mathcal{S}(v, s, t)$ is the set of sale items of variant v in store s during period t , and q_i is the quantity of sale item i .

A period without sales is treated as zero demand only from the date on which the variant was first stocked in that store, because demand before that date is unknown, not zero.

Size-level series are sparse and intermittent. Each series is therefore classified using the Syntetos–Boylan scheme, based on the average inter-demand interval (ADI) and the squared coefficient of variation of non-zero demand (CV^2), as smooth, erratic, intermittent, or lumpy (commonly with thresholds $ADI = 1.32$ and $CV^2 = 0.49$). This helps determine which baseline methods are used and makes it possible to report accuracy by demand type.

During a stock-out, observed sales underestimate demand. If stock history is available, weeks in which the variant was unavailable for most of the week are marked as censored and given zero weight in the training loss, instead of being learned as true zero demand.

5.5 Feature Engineering

Each training example describes one variant in one store at a forecast origin t , with the demand at $t + h$ as the target. The feature groups are:

- **Lag features:** demand at lags of 1 to 8 weeks, and at 52 weeks if at least one year of history exists (recent level, yearly seasonality).
- **Rolling statistics:** mean, standard deviation, and share of zero-sale weeks over the past 4, 8, and 12 weeks (level, volatility, intermittency).
- **Calendar features:** week of year, month, and holiday and festival indicators (seasonality, events).
- **Price features:** selling price, discount depth, and price relative to the category average (price effect).
- **Product and store attributes:** category, size, colour, store, and city, used as categorical features (sharing patterns across similar series).
- **Lifecycle features:** weeks since first sale and weeks since last sale (product age).

To prevent data leakage, features at origin t use only data available up to t . Only features that are known in advance, such as calendar events, may refer to the future.

5.6 Demand Forecasting Model

5.6.1 Global Gradient-Boosted Model

The individual size-level series are too short and sparse to support one model per series. Following the global models of Montero-Manso and Hyndman and the forecasting of groups of similar series by Bandara et al., StockSense builds one global model per tenant on all of its series, with product, size, and store as categorical features. The chosen model is LightGBM (gradient-boosted decision trees). Gradient-boosted trees performed strongly against several statistical methods in the M5 competition (Makridakis et al.), and LightGBM handles missing values and categorical variables natively and trains quickly on tabular data.

5.6.2 Loss Function and Forecast Horizon

Demand is non-negative and contains many zeros, so squared error is not a suitable loss. The model uses the Tweedie objective (power between 1 and 2), which models a point mass at zero together with a positive continuous part. The Poisson objective and squared error are retained as alternatives and compared experimentally.

Multi-step forecasts use a direct approach: a separate model is trained for each step $h = 1, \dots, H$ of the forecast horizon, so errors do not

accumulate through recursive predictions:

$$\hat{y}(v, s, t + h) = f_h(\mathbf{x}_{v,s,t}), \quad (5.2)$$

where $\mathbf{x}_{v,s,t}$ denotes the features of variant v in store s at time t .

5.6.3 Training and Tuning

The hyperparameters (number of leaves, learning rate, minimum data per leaf, feature fraction, bagging fraction, and boosting rounds) are tuned with rolling-origin validation, using an expanding training window and early stopping on each fold. Random shuffling is never used, because it would expose future data to the model. Sample weights exclude censored periods from the loss.

5.6.4 Statistical Baselines

To evaluate whether the more complex model is necessary, the following baselines are computed on the same splits: the naive forecast (last value), the seasonal naive forecast (with at least one year of history), the moving average, Holt–Winters exponential smoothing (`statsmodels`), and Croston’s method for intermittent series.

5.6.5 Initial Training Period

StockSense does not ship with a pre-trained model, and the data of one tenant is never shared with another. A new tenant therefore starts without a forecasting model. The owner first records sales through the POS or uploads historical sales reports. Until a minimum amount of history is available (for example, several weeks of sales for the tenant’s products), no model-based forecast is displayed and the user is told that more data is needed (UC-10). During this period, simple baseline forecasts may be shown if enough observations exist. Once the minimum is reached, the first global model is trained on the tenant’s own data, and model-based forecasting and recommendations start working. The model is then retrained on a schedule as new sales arrive; accuracy is measured rather than assumed to improve as history grows. The minimum history is determined experimentally (Section 5.10).

5.6.6 Forecast Uncertainty

In addition to the point forecast, an upper-quantile forecast (for example, the 0.9 quantile using the quantile objective of LightGBM) is trained. The recommendation engine uses it to establish safety stock, so that decisions account for uncertainty and not only the mean.

5.7 Recommendation Engine and Stock Redistribution

For every variant v and store s , the engine compares the forecast with the stock position over a planning horizon of H weeks. Define:

- I = on-hand stock plus pending incoming transfers (pending transfers are already accepted, so they are not recommended again),
- D = forecast demand over the next H weeks,
- $Q_{0.9}$ = the upper-quantile forecast over the horizon, and $SS = \max(0, Q_{0.9} - D)$ = safety stock above the expected demand.

The net position is therefore

$$N = I - (D + SS). \quad (5.3)$$

If $N > 0$, the store has a surplus of N units; if $N < 0$, it has a shortage of $-N$ units.

Redistribution. Following the proactive-transshipment approach of Paterson et al., transfers are planned before a stock-out or dead stock

occurs. For each variant, the transfer quantities x_{ij} from surplus store i to shortage store j are obtained by solving a transportation problem:

$$\max_x \sum_i \sum_j (m - c_{ij}) x_{ij} \quad (5.4)$$

$$\text{s.t. } \sum_j x_{ij} \leq \text{surplus}_i \quad \forall i, \quad (5.5)$$

$$\sum_i x_{ij} \leq \text{shortage}_j \quad \forall j, \quad (5.6)$$

$$x_{ij} \geq 0. \quad (5.7)$$

Here m_{ij} is the relevant unit margin at destination store j and c_{ij} is the unit transfer cost between stores i and j . A transfer is recommended only when $m_{ij} - c_{ij} > 0$. Transfer quantities are constrained to non-negative integer units; because the transportation matrix and the supply and demand values are integral, the linear-program solution has an integral optimum. Since retailers have few branches, each problem is small and cheap to solve. The model does not yet include transfer lead time or capacity limits; these are recorded as assumptions for the backtest.

Reorder. Any shortage remaining after redistribution is converted into a reorder recommendation with a suggested quantity.

Priority. Recommendations are processed in the order redistribution, markdown, reorder, because moving existing stock is usually easier and less costly than discounting or reordering.

5.8 Markdown Optimization

Any surplus that is not cleared by redistribution is passed to the markdown optimizer. Like Caro and Gallien, it follows a forecast-then-optimize structure, but it is used as an early-warning mechanism rather than an end-of-season clearance approach.

A variant-store is marked as at risk when its weeks of cover exceeds the remaining selling window. For positive forecast demand, this is calculated as:

$$\text{WOC} = \frac{I}{\bar{d}} > T, \quad (5.8)$$

where $\bar{d}$ is the average forecast weekly demand and T is the number of weeks in which the stock should be sold. If $\bar{d} = 0$ and $I > 0$, the item is treated as at risk and handled separately because its weeks of cover is undefined. The candidate discounts are evaluated against expected margin and sell-through; the smallest discount meeting the target is selected only when it does not produce a lower expected margin than the no-discount case.

For each candidate discount d from a grid (for example, 5% to 50% in steps of 5%), demand is adjusted using a constant price-elasticity model:

$$D(d) = D \cdot (1 - d)^{-\varepsilon}, \quad \text{Sold}(d) = \min(I, D(d)), \quad (5.9)$$

where $D(d)$ is the total adjusted demand over the horizon.

The system recommends the smallest discount for which the projected sell-through reaches a target, subject to a margin floor $p(1 - d) \geq c$, where p is the selling price and c is the cost price. The estimated impact is the difference in margin with and without the discount:

$$\text{Impact} = \text{Sold}(d) (p(1 - d) - c) - \text{Sold}(0) (p - c). \quad (5.10)$$

The elasticity ε is estimated by log-log regression on the tenant's own price history when there is sufficient price variation. Otherwise, a conservative category-level default is used, and the recommendation states that the estimate is approximate.

5.9 Explainable Recommendations

Every recommendation is stored together with a two-part explanation:

- A structured record of the quantities involved in the decision: forecast demand, current stock, net position, weeks of cover, source and destination stores, margin, transfer cost, and estimated impact.
- The main forecast drivers, extracted from the per-feature contributions of LightGBM (`pred_contrib`, additive SHAP-style values)

and rendered as plain phrases such as “recent weekly sales” or “seasonal pattern”.

Both parts are filled into text templates, for example: “Size M of this product is expected to sell 12 units at Store B in the next 4 weeks, but only 3 are in stock, while Store A holds 15 units that are expected to sell only 4.” The explanation is saved at creation time, so it always shows the situation on which the recommendation was based.

5.10 Forecast and Recommendation Evaluation

5.10.1 Data

Public retail datasets are used for initial development and validation. A dataset such as M5 contains store-item sales and prices, which supports evaluation at the store level and the hierarchy level, but it does not contain size-level sales. Size-level behaviour is therefore evaluated on real retailer sales data if access is secured, and the findings from public data are interpreted with this limitation in mind, as stated in Chapter 2.

5.10.2 Protocol

All splits are chronological. Hyperparameters are tuned with rolling-origin validation, and the final comparison is made on the most recent period, which the models never see during training or tuning.

5.10.3 Metrics

The primary metrics are

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{k=1}^n (a_k - f_k)^2}, \quad (5.11)$$

$$\text{MAPE} = \frac{100}{|P|} \sum_{k \in P} \frac{|a_k - f_k|}{a_k}, \quad P = \{k : a_k > 0\}, \quad (5.12)$$

where a_k is the actual demand and f_k is the forecast.

Size-level series contain many zeros and MAPE is undefined for zero actuals, so non-zero-demand MAPE is computed only on periods with positive actual demand, together with the proportion of periods excluded. WAPE (total absolute error divided by total actual sales) and MAE are emphasized as supplementary measures, since they remain valid for sparse series. Results are presented separately for each demand class, for fast and slow movers.

5.10.4 Experiments

- LightGBM against the statistical baselines (the main claim of Chapter 3).
- The global model against separate models for each series.
- Forecast accuracy against the amount of training history available (for example, 4, 8, and 12 weeks), to estimate how much data a new tenant must provide before forecasts become reliable.
- Tweedie and Poisson objectives against squared error.

5.10.5 Recommendation Backtest

The value of the recommendations is estimated by replaying history. At each cut-off date, the engine produces recommendations using only the information available up to that date. The recommended transfers and discounts are applied to a simulated stock record, and the following weeks are replayed with actual sales as demand. The simulation specifies transfer lead time, assumes that stock is unavailable at the source while it is in transit, and applies an accepted markdown from its effective date. The outcomes are compared with a “no action” run and include dead stock (units whose cover is greater than T weeks), units of lost sales caused by stock-outs, and the sell-through rate. The simulation assumes that transfers do not change demand, and that demand changes only through the elasticity model when a markdown is applied. These assumptions are stated together with the results.

5.11 Iterative Development

StockSense is built in increments, and each increment ends with the testing activities described in Chapter 6 and a review by the supervisor:

1. **Multi-tenant foundation:** tenant and user management, products, inventory, POS, and the ingestion and validation pipeline.
2. **Forecasting experiments** in Jupyter Notebook: feature engineering, baselines, and LightGBM, all evaluated offline.
3. **Integration:** scheduled batch forecasting, the recommendation engine, and redistribution.
4. **Markdown optimization,** explanations, notifications, and the dashboard.

Building the forecasting models offline first allows the choice of model to be checked on data before integration.

5.12 Conclusion

The methodology treats each sale as the beginning of a decision path. Data is validated and standardized, and demand is modelled at the product-size-store level using a global LightGBM model that is compared against statistical baselines. A new tenant first provides sales data so that a model can be trained on its own history, after which forecasting starts. The recommendation engine then transforms forecasts into actions in a fixed order: redistribution through a small transportation problem, early markdown through a sell-through model, and finally reorder for the remaining shortage. Every recommendation includes an explanation and an estimate of its impact. The evaluation uses chronological splits and a replay of past data, so that both forecast accuracy and the potential reduction of dead stock can be measured.

Chapter 6 High-Level and Low-Level Design

6.1 System Overview

StockSense is a multi-tenant SaaS solution for small and medium-sized retailers to minimise dead stock and stock out in their stores, leveraging their own sales data. Every business is an independent tenant and has its own users, stores, products, inventory and sales. The sales are made using the built-in POS system or via CSV/Excel files which can be uploaded. The system predicts demand at product-size-store level and suggests actions with an accompanying clear explanation on a single web dashboard. These will be the modules that will be incorporated into our system:

6.1.1 Tenant and User Management

This module processes the registration of a business, user logon and roles. Ensures that an owner, store manager and/or administrator can only access information for which they have the authority and tenant.

6.1.2 POS and Inventory Management

This module is the free part of StockSense. It records sales and keeps the current stock of every product, size, and store up to date.

6.1.3 Data Ingestion and Cleaning

This module imports historical sales data from CSV or Excel files, validates the data, identifies the invalid records and formats the data into a consistent standard format. It allows businesses that already have a point of sale or spreadsheet to participate without having to change their system.

6.1.4 Demand Forecasting

This module forecasts the weekly and monthly demand of each product, size and store. It trains a model on each tenant's own sales history, so a new tenant must first record or upload enough sales data before forecasts are shown.

6.1.5 Stock Redistribution and Reorder Recommendations

This module is a comparison of the forecast and the current stock of each branch. It recommends moving stock from a branch with more stock to one with less stock first and suggests a reorder only if the amount of stock is insufficient being moved.

6.1.6 Markdown Optimization

The module identifies slow moving stock and suggests selling the stock at an early stage prior to it being dead.

6.1.7 Explainable Recommendations

Each recommendation is accompanied by a brief description of why it is recommended, and an approximate financial impact so the owner can comprehend and commit to the recommendation.

6.1.8 Dashboard and Reporting

This module displays the sales, inventory, forecasts, and recommendations to the Owner and the Store Managers in an easy to use web interface.

6.2 Design Considerations

This section lists the dependencies and limitations that affect the design of StockSense, so that they can be handled before the implementation starts.

6.2.1 Assumptions and Dependencies

6.2.1.1 Operating System and Browser

StockSense is a web application. Users must be running a relatively new version of Chrome, Edge, Firefox or Safari on their laptop, desktop or tablet.

6.2.1.2 End User Characteristics

- Users have a stable internet connection.
- Owners/managers have not mastered computers but know some of the basic uses.
- The retailer can access historical sales data or can record new sales in the built-in, point-of-sale system.

6.2.1.3 Possible Changes in Functionality

The quality of forecasts depends on the quality and amount of sales data. During development, the functionality may change as a result of testing of the datasets and feedback from the supervisor. That is the intent of the modular design to facilitate such change.

6.2.2 General Constraints

6.2.2.1 Hardware and Software Environment

- More computing power is required for forecasting and data processing (than for normal page requests).
- The interface needs to be compatible with various devices and browsers.
- There may be constraints on what we can do when implementing with external libraries or service.

6.2.2.2 End-User Environment

- Slow or unstable internet in some shops.
- Many managers have no training in data analysis and the interface should not be technologic.

6.2.2.3 Data Format Differences

- Uploaded files vary in column names, date formatting and product naming.
- All of them are to be converted to a single standard internal format.

6.2.2.4 Data Repository and Tenant Isolation

- Never any one business should have access to another business data.
- All records should be associated with tenant.
- The database needs to expand to include additional tenants and additional sales history.

6.2.2.5 Security Requirements

- It is user's responsibility to log in with their own username and password, and to keep their password protected.
- Access should be based on user role.
- Data needs to be safeguarded whilst in transit and whilst stored.

6.2.2.6 Performance Requirements

- Forecasting and large uploads have to be performed in the background.
- The results with frequent use, like the latest forecasts, are stored and do not need to be calculated time and time again.
- Clearly error messages should be produced for invalid input.

6.2.2.7 Interoperability Requirements

- The system needs to be able to import CSV and Excel files from external point-of-sales (POS) systems and spreadsheets.
- The internal modules should communicate using unambiguous interfaces.

6.2.2.8 Verification and Validation Requirements

- It is important to test the upload pipeline with bad and garbled files.
- Tenant separation is necessary to be tested for data leakage.
- Predictions need to be judged against past data, as described in Chapter 5.

6.2.3 Goals and Guidelines

6.2.3.1 The KISS Principle

The design should be kept simple to make maintenance and non-technical user friendly.

6.2.3.2 Separation of Tenant Data

Each firm requires its very own safeguarded data room.

6.2.3.3 Modularity and Extensibility

Forecasting, recommendations, POS, ingestion, and markdown should be separated so as to allow the inclusion of new forecasting approaches without reengineering the system.

6.2.3.4 Explainability

All recommendations should include the rationale for the recommendation.

6.2.4 Development Methods

StockSense will be developed iteratively, in stages. Initial modules will include tenant management, product and inventory management and data ingestion, followed by forecasting, recommendations, markdown optimization and dashboard. An evaluation will be conducted for each stage prior to the beginning of the next stage and the plan will be modified based on feedback from the supervisor.

6.3 System Architecture

StockSense has a multi-layered web structure. The users interact with the web dashboard or the POS interface, which will communicate with the API in the back end. The API verifies the identity of the user, their tenant and passes the request to the appropriate module. The cleaned sales data feeds into a forecasting engine, while the forecasts are fed into a recommendation engine and the valid stock data from the POS and inventory module feeds into a recommendation engine. The slow moving stock is passed to the markdown optimizer. All the results are recorded in the database and displayed back to the user at the dashboard. The full picture is displayed in Figure 6.1.

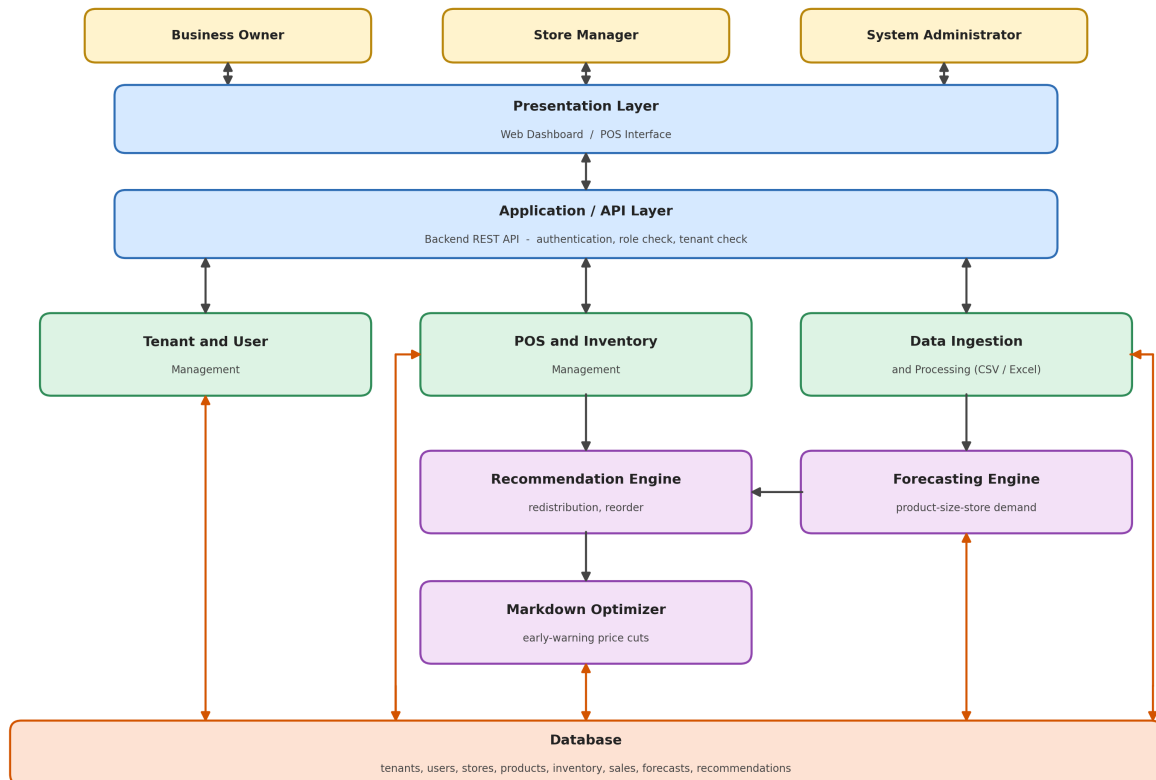


Figure 6.1: System Architecture Diagram
This figure shows the System Architecture Diagram of our Application

6.3.1 Subsystem Architecture

6.3.1.1 User and Tenant Management

This subsystem contains business and business users. It supports roles, login and registration, and ensures that every user only has access to data from his/her tenant.

6.3.1.2 POS and Inventory Subsystem

This subsystem maintains the current item count for each item, size and store, and documents the total sales for each item, size, store.

6.3.1.3 Data Ingestion Subsystem

This subsystem is able to accept CSV and Excel files, validate them, flag invalid records and convert files to the standard format used internally.

6.3.1.4 Forecasting Subsystem

This subsystem analyzes the sales history of a tenant and forecasts the weekly and monthly demand at product-size-store level. It trains a model on the tenant's own data, and no forecast is shown until enough history has been recorded or uploaded.

6.3.1.5 Recommendation Subsystem

This subsystem is a comparison of forecasts with the level of each branch. It advises to move stock from one branch to another first, and then recommends a reorder when this is not sufficient. These recommendations along with the reason and estimated financial impact are stored with each recommendation.

6.3.1.6 Markdown Optimization Subsystem

This subsystem is looking for stock that is not selling fast enough and advises a reduction in price early enough that it can be sold.

6.4 Architectural Strategies

6.4.1 Multi-Tenant Architecture

Many companies will share the same platform but the data will remain logically separate. All records that are important will have a tenant identifier on them, and all requests will be compared to it before reading or updating any data.

6.4.2 Modular Architecture

The modules will be forecasting, recommendations, POS, data ingestion, markdown optimization. If one of them changes (e.g. a new forecasting model), then the others should not need to change significantly.

6.4.3 Programming Languages and Frameworks

The web interface will be created using React. The backend REST API will be developed in python (FastAPI or Django), the main model for forecasting will be lightgbm, the statistical baseline will be statsmodels and pandas and openpyxl will be used for processing of csv and excel files.

6.4.4 Database

A PostgreSQL relational database will store all business data. Each table in a business has a business tenant identifier, which is helpful for tenant isolation covered in Section 6.4.1. The detailed design will be provided in the database design part of chapter 4.

6.4.5 Background Processing

Forecasting and cleaning of large files may be time-consuming. These jobs will continue to run in the background and the user will be told when the results are available so the dashboard remains speedy.

6.4.6 Error Detection and Recovery

The system will record the following information regarding failed uploads, rejected records and failed forecasting runs to help identify the cause: In case a forecasting run is unsuccessful, previous valid forecasts will remain and not be eliminated from the forecasting dashboard.

6.5 Domain Model/Class Diagram

The domain model of StockSense is shown in Figure 6.2. It follows the data dictionary of Chapter 4: every business-owned class belongs to exactly one **Tenant**, and the foreign keys of the database are expressed as associations between classes.

The major classes of the system are:

- **Tenant and Subscription**: a business on the platform and its free/paid plan history.
- **User**: Business Owner, Store Manager, or System Administrator (distinguished by the `role` attribute).
- **Store**: a branch of a business.
- **Product and ProductVariant**: a product and its sizes/variants, each with its own SKU.
- **Inventory**: the quantity of one variant held at one store.
- **Sale and SaleItem**: a customer sale (POS or uploaded) and the variants it contains.
- **UploadJob**: an uploaded CSV/Excel file with its accepted and rejected row counts.
- **Forecast**: the predicted demand of one variant at one store for a weekly or monthly period.

- The basic relationships between these classes are as follows:

- StockSense - Domain Model / Class Diagram**

```

classDiagram
    class Subscription {
        - subscriptionID : Integer
        - plan : String
        - startDate : Date
        - endDate : Date
        + upgrade()
        + downgrade()
    }
    class Tenant {
        - tenantID : Integer
        - businessName : String
        - status : String
        - createdAt : Timestamp
        + suspend()
        + reactivate()
    }
    class User {
        - userID : Integer
        - name : String
        - email : String
        - passwordHash : String
        - role : String
        - isActive : Boolean
        + login()
        + logout()
    }
    class Store {
        - storeID : Integer
        - name : String
        - city : String
        - address : String
        - isActive : Boolean
    }
    class Product {
        - productID : Integer
        - name : String
        - category : String
        - costPrice : Decimal
        - sellingPrice : Decimal
    }
    class Inventory {
        - inventoryID : Integer
        - quantity : Integer
        - updatedAt : Timestamp
        + adjustQuantity()
        + reduceStock()
    }
    class Notification {
        - notificationID : Integer
        - message : String
        - isRead : Boolean
        - createdAt : Timestamp
        + markAsRead()
    }
    class StockTransfer {
        - transferID : Integer
        - quantity : Integer
        - status : String
        + complete()
    }
    class Recommendation {
        - recommendationID : Integer
        - explanation : Text
        - estimatedImpact : Decimal
        - status : String
        - decidedAt : Timestamp
        - createdAt : Timestamp
        + accept()
        + dismiss()
    }
    class RedistributionRecommendation {
        - quantity : Integer
        + createStockTransfer()
    }
    class Forecast {
        - forecastID : Integer
        - periodStart : Date
        - horizon : String
        - predictedQuantity : Decimal
        - modelName : String
        - generatedAt : Timestamp
        + generate()
    }
    class ProductVariant {
        - variantID : Integer
        - sku : String
        - size : String
        - color : String
        - currentPrice : Decimal
    }
    class Sale {
        - saleID : Integer
        - saleDate : Timestamp
        - source : String
        - totalAmount : Decimal
        + recordSale()
    }
    class SaleItem {
        - saleItemID : Integer
        - quantity : Integer
        - unitPrice : Decimal
    }
    class UploadJob {
        - uploadID : Integer
        - fileName : String
        - rowCreated : Integer
        - rowAccepted : Integer
        - rowRejected : Integer
        - status : String
        - uploadDate : Timestamp
        + validateRow()
        + normalizeRow()
    }

    Subscription "1" -- "1..*" Tenant : has
    Tenant "1" -- "1..*" User : employs
    Tenant "1" -- "0..1" Store : owns
    Tenant "1" -- "0..1" Product : collects
    Tenant "1" -- "0..1" UploadJob : uploads
    User "1" -- "0..1" Store : assigned to
    User "1" -- "0..1" Notification : receives
    Store "1" -- "1..*" Inventory : to store from here
    Store "1" -- "0..1" Recommendation : source store
    Store "1" -- "0..1" Forecast : forecast for
    Store "1" -- "0..1" ProductVariant : stocked at
    Store "1" -- "0..1" Sale : imports
    Product "1" -- "0..1" ProductVariant : 1..*
    Product "1" -- "0..1" Sale : 0..1
    Inventory "0..1" -- "1" ProductVariant : forecast of
    Inventory "0..1" -- "0..1" Forecast : concerns
    Notification "0..1" -- "0..1" StockTransfer : raised
    Notification "0..1" -- "0..1" Recommendation : triggers
    StockTransfer "0..1" -- "0..1" Recommendation : destination store
    StockTransfer "0..1" -- "0..1" Forecast : basis of
    Recommendation "0..1" -- "0..1" Forecast : basis of
    Recommendation "0..1" -- "0..1" ProductVariant : sold at
    Recommendation "0..1" -- "0..1" Sale : contains
    Recommendation "0..1" -- "0..1" RedistributionRecommendation : (creates)
    Recommendation "0..1" -- "0..1" ReorderRecommendation
    Recommendation "0..1" -- "0..1" MarkdownRecommendation
    Forecast "0..1" -- "0..1" Recommendation : (creates)
    Forecast "0..1" -- "0..1" ProductVariant : forecast of
    Forecast "0..1" -- "0..1" Sale : contains
    ProductVariant "1" -- "0..1" Sale : sold at
    ProductVariant "1" -- "0..1" SaleItem : contains
    Sale "1" -- "0..1" SaleItem : contains
    UploadJob "0..1" -- "0..1" ProductVariant : imports
  
```

Subscription

 - subscriptionID : Integer
 - plan : String
 - startDate : Date
 - endDate : Date
 - + upgrade()
 - + downgrade()

Tenant

 - tenantID : Integer
 - businessName : String
 - status : String
 - createdAt : Timestamp
 - + suspend()
 - + reactivate()

User

 - userID : Integer
 - name : String
 - email : String
 - passwordHash : String
 - role : String
 - isActive : Boolean
 - + login()
 - + logout()

Store

 - storeID : Integer
 - name : String
 - city : String
 - address : String
 - isActive : Boolean

Product

 - productID : Integer
 - name : String
 - category : String
 - costPrice : Decimal
 - sellingPrice : Decimal

Inventory

 - inventoryID : Integer
 - quantity : Integer
 - updatedAt : Timestamp
 - + adjustQuantity()
 - + reduceStock()

Notification

 - notificationID : Integer
 - message : String
 - isRead : Boolean
 - createdAt : Timestamp
 - + markAsRead()

StockTransfer

 - transferID : Integer
 - quantity : Integer
 - status : String
 - + complete()

Recommendation

 - recommendationID : Integer
 - explanation : Text
 - estimatedImpact : Decimal
 - status : String
 - decidedAt : Timestamp
 - createdAt : Timestamp
 - + accept()
 - + dismiss()

RedistributionRecommendation

 - quantity : Integer
 - + createStockTransfer()

Forecast

 - forecastID : Integer
 - periodStart : Date
 - horizon : String
 - predictedQuantity : Decimal
 - modelName : String
 - generatedAt : Timestamp
 - + generate()

ProductVariant

 - variantID : Integer
 - sku : String
 - size : String
 - color : String
 - currentPrice : Decimal

Sale

 - saleID : Integer
 - saleDate : Timestamp
 - source : String
 - totalAmount : Decimal
 - + recordSale()

SaleItem

 - saleItemID : Integer
 - quantity : Integer
 - unitPrice : Decimal

UploadJob

 - uploadID : Integer
 - fileName : String
 - rowCreated : Integer
 - rowAccepted : Integer
 - rowRejected : Integer
 - status : String
 - uploadDate : Timestamp
 - + validateRow()
 - + normalizeRow()

Notes:

 - Every business-owned class belongs to exactly one Tenant (TenantID in the database).
 - Foreign keys are shown as associations, not as attributes.
 - Recommendation is stored in one table. Its type returns Redistribution / Reorder / Markdown corresponds to the time subscription.
 - Admin users have no Tenant (TenantID is 0). Store Manager users are assigned to a Store.

Figure 6.2: Domain Model / Class Diagram
This figure shows the Domain Model / Class Diagram of our Application

6.6 Sequence Diagrams

This section presents one sequence diagram for each use case described in Chapter 4. Each diagram shows the order of messages exchanged between the actor, the user interface, the application layer (API / controller or service), and the database for the basic flow of the use case. Alternative flows are drawn as **alt** fragments, optional behavior as **opt** fragments, repeated steps as **loop** fragments, and operations that must succeed or fail together as a single database transaction.

6.6.1 UC-01: Register Business

This diagram shows the interaction for the use case **Register Business** (UC-01).

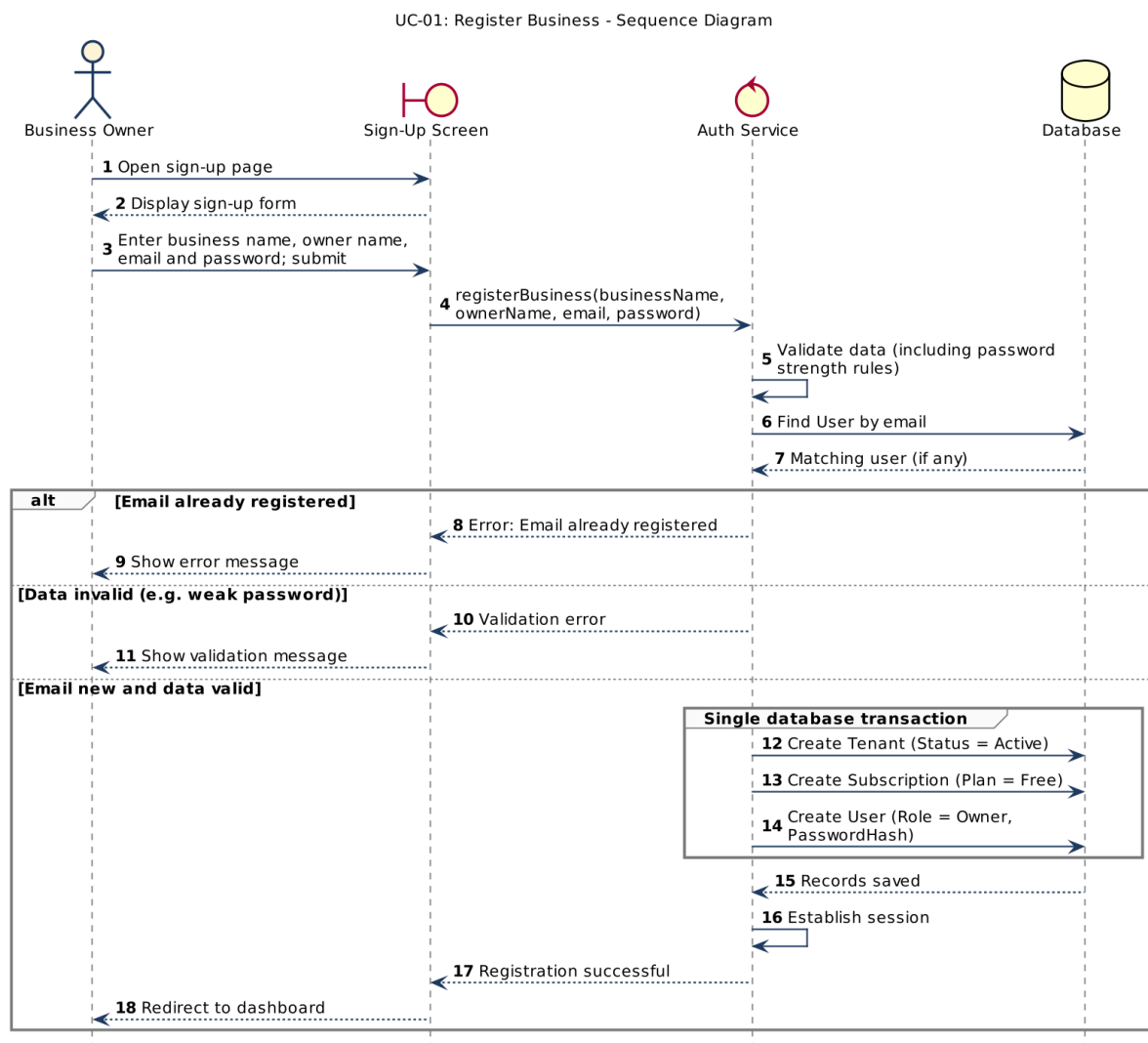


Figure 6.3: Sequence Diagram: Register Business
This figure shows the sequence diagram of UC-01 (Register Business)

6.6.2 UC-02: Login

This diagram shows the interaction for the use case **Login** (UC-02).

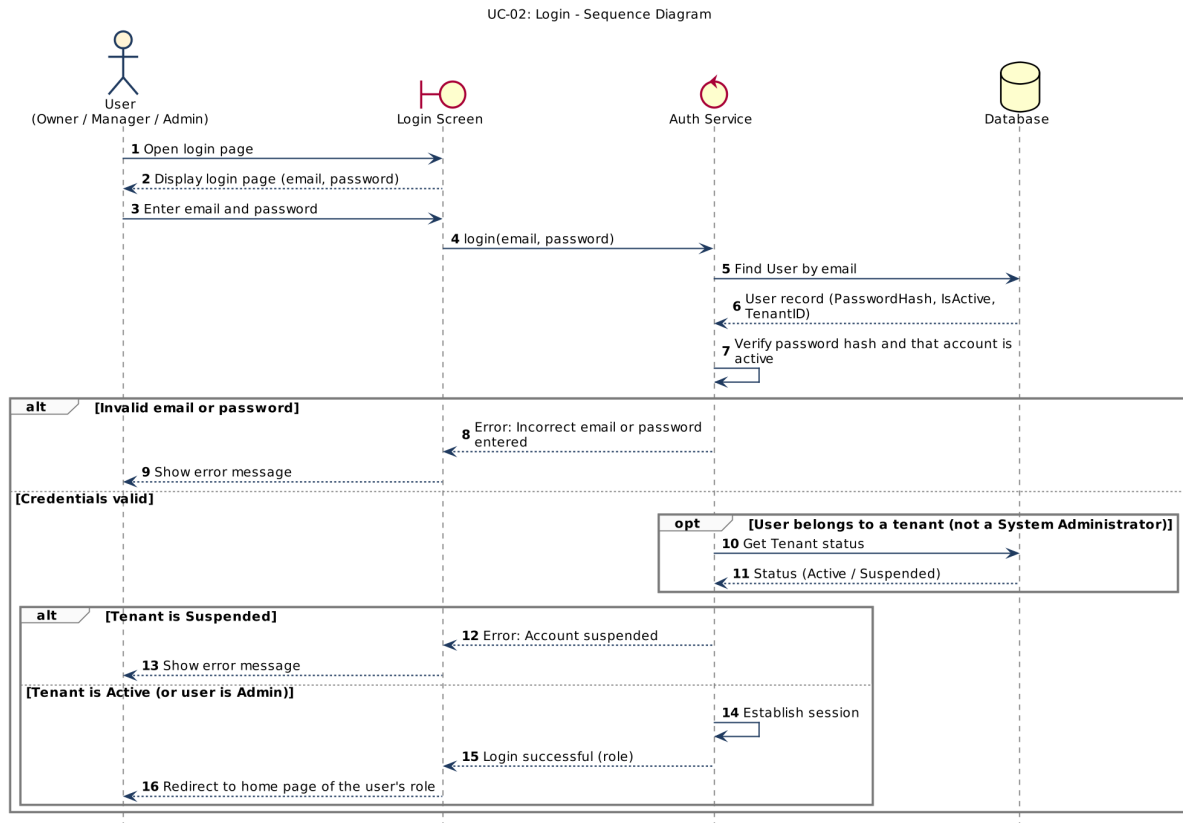


Figure 6.4: Sequence Diagram: Login
This figure shows the sequence diagram of UC-02 (Login)

6.6.3 UC-03: Manage Stores

This diagram shows the interaction for the use case **Manage Stores** (UC-03).

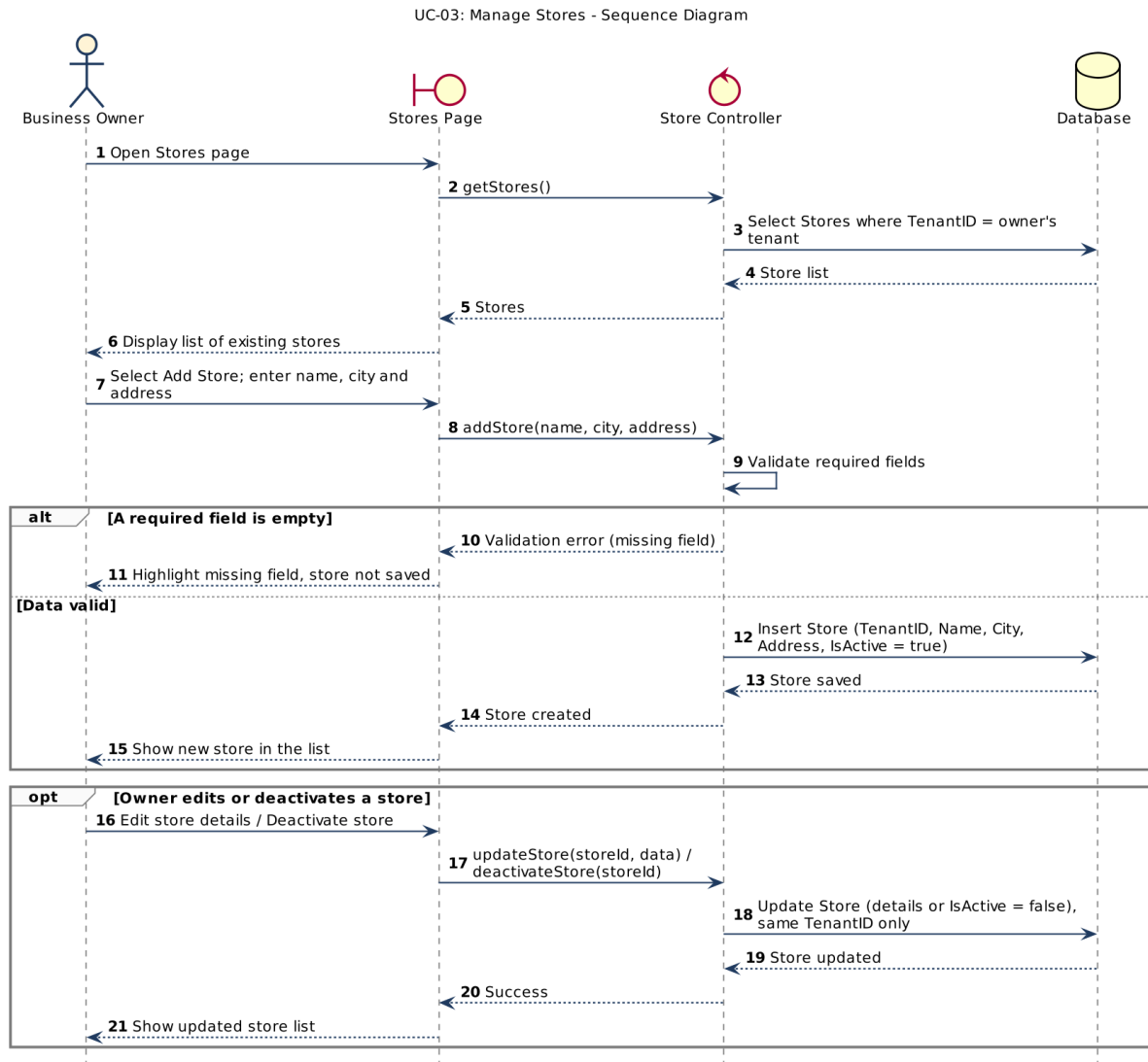


Figure 6.5: Sequence Diagram: Manage Stores
 This figure shows the sequence diagram of UC-03 (Manage Stores)

6.6.4 UC-04: Manage Store Managers

This diagram shows the interaction for the use case **Manage Store Managers** (UC-04).

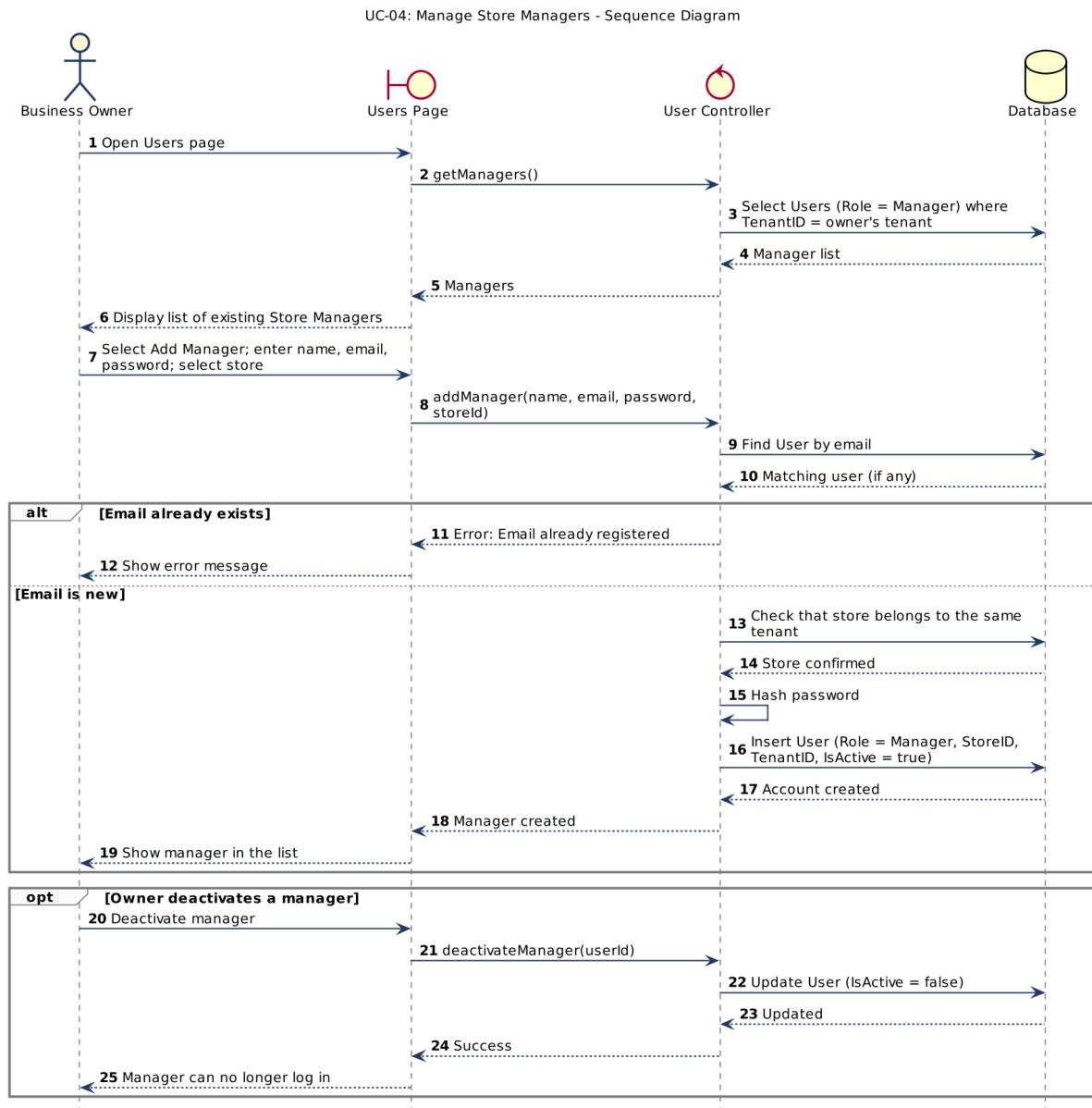


Figure 6.6: Sequence Diagram: Manage Store Managers
 This figure shows the sequence diagram of UC-04 (Manage Store Managers)

6.6.5 UC-05: Manage Products and Variants

This diagram shows the interaction for the use case **Manage Products and Variants** (UC-05).

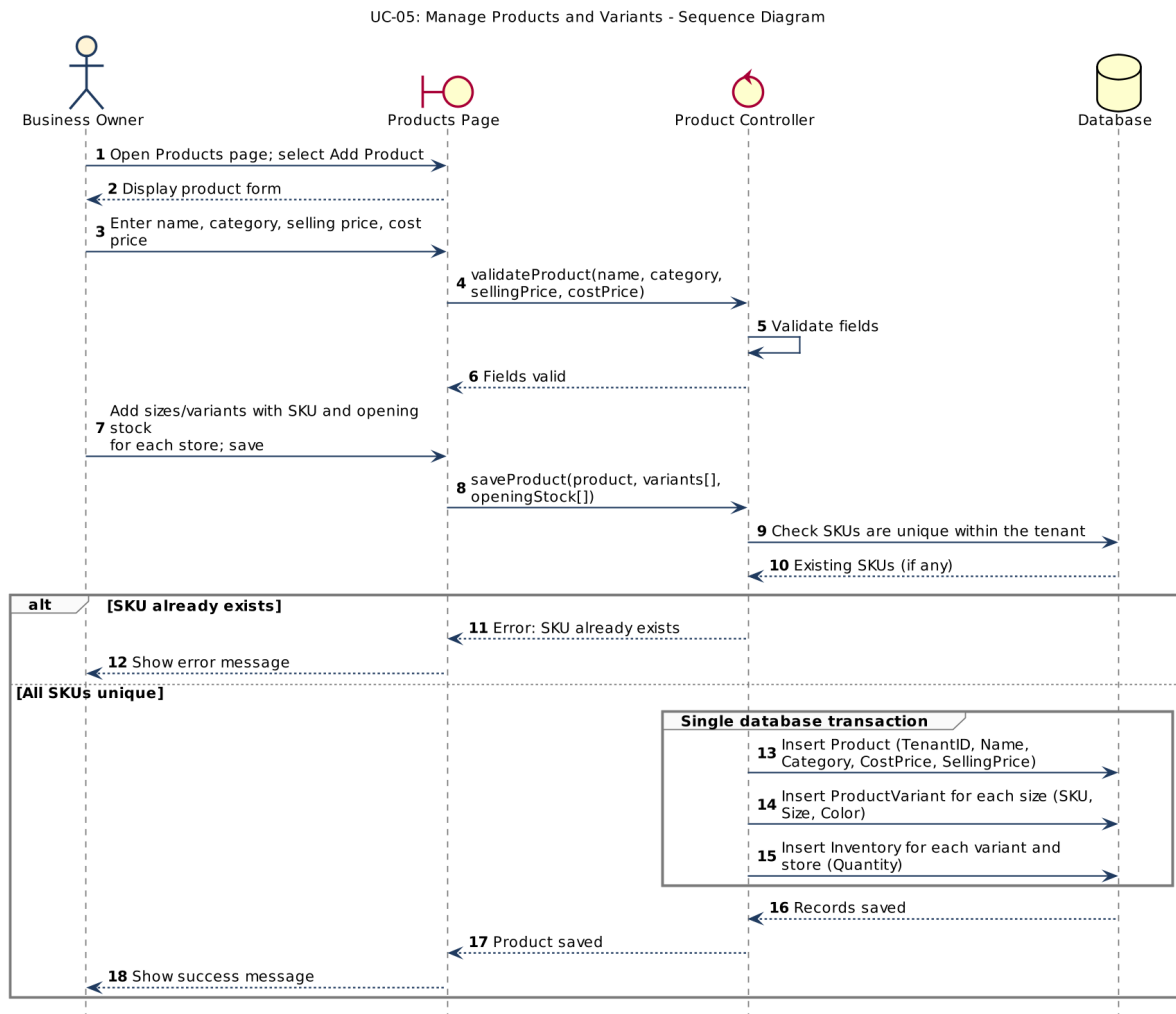


Figure 6.7: Sequence Diagram: Manage Products and Variants
 This figure shows the sequence diagram of UC-05 (Manage Products and Variants)

6.6.6 UC-06: Adjust Inventory

This diagram shows the interaction for the use case **Adjust Inventory** (UC-06).

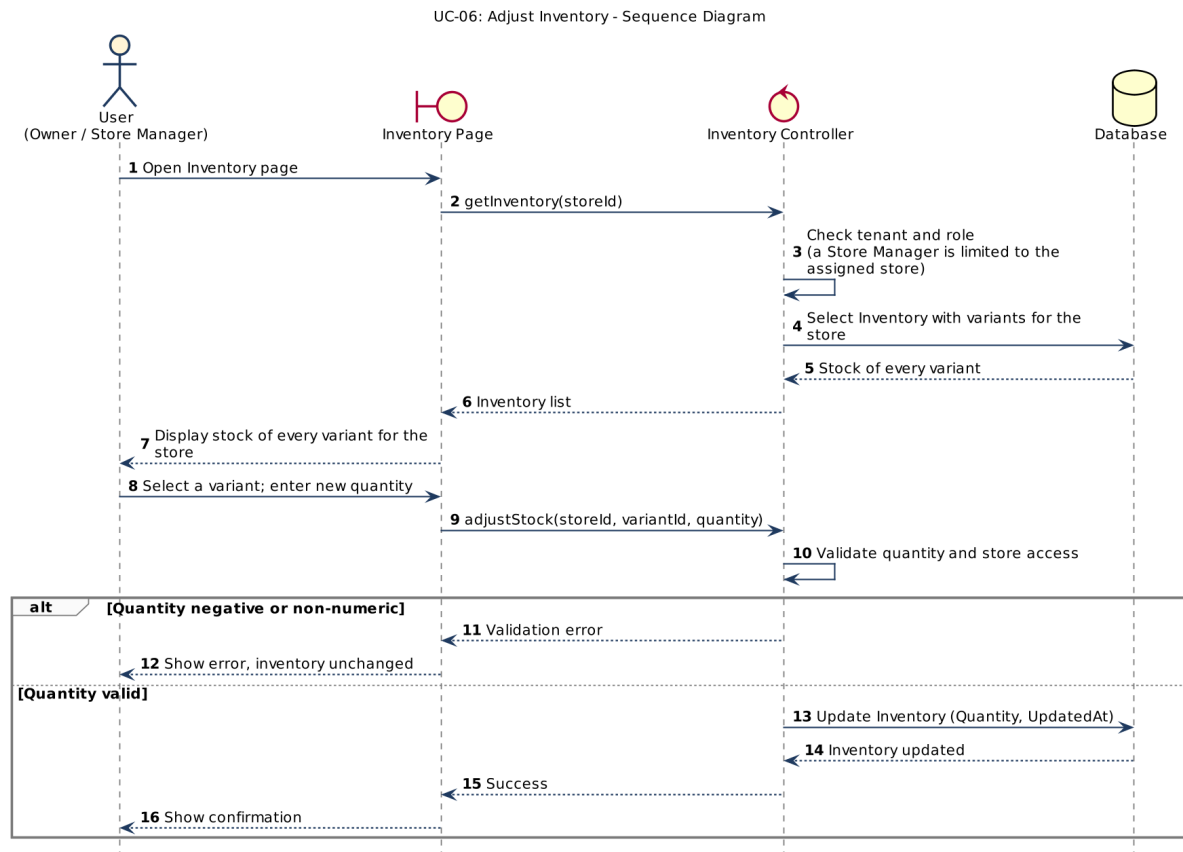


Figure 6.8: Sequence Diagram: Adjust Inventory
 This figure shows the sequence diagram of UC-06 (Adjust Inventory)

6.6.7 UC-07: Record Sale (POS)

This diagram shows the interaction for the use case **Record Sale (POS)** (UC-07).

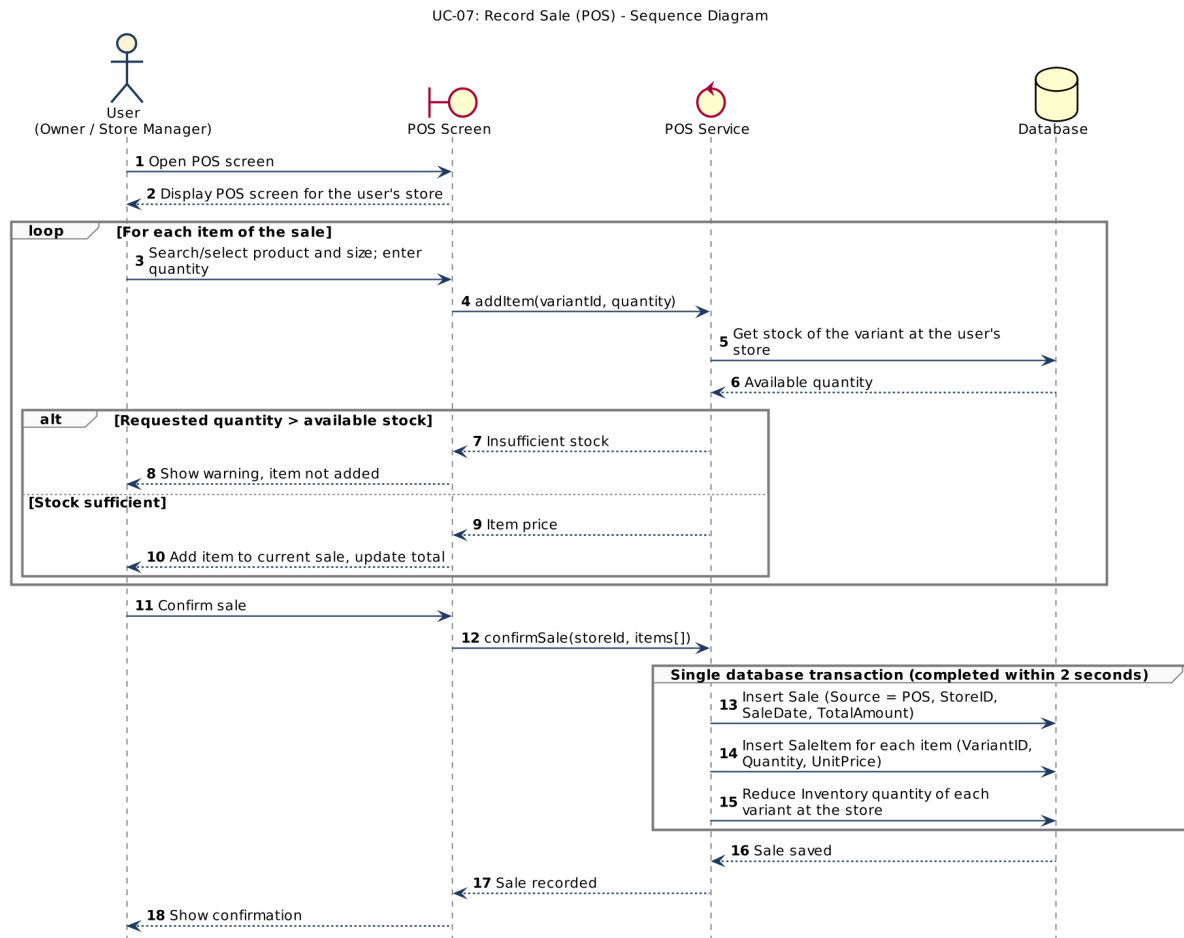


Figure 6.9: Sequence Diagram: Record Sale (POS)
 This figure shows the sequence diagram of UC-07 (Record Sale (POS))

6.6.8 UC-08: Upload Historical Sales Data

This diagram shows the interaction for the use case **Upload Historical Sales Data** (UC-08).

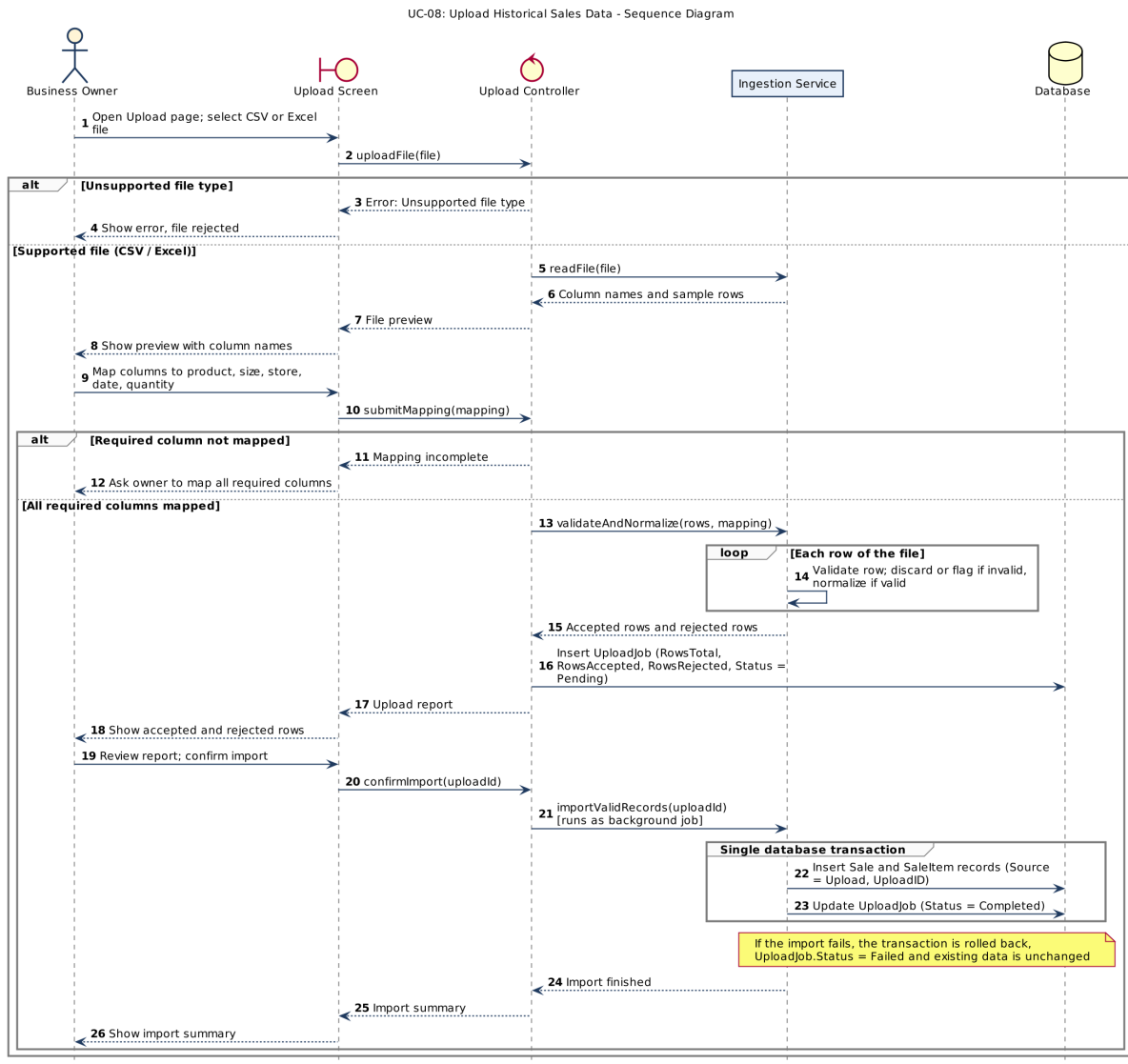


Figure 6.10: Sequence Diagram: Upload Historical Sales Data
 This figure shows the sequence diagram of UC-08 (Upload Historical Sales Data)

6.6.9 UC-09: View Dashboard and Alerts

This diagram shows the interaction for the use case **View Dashboard and Alerts** (UC-09).

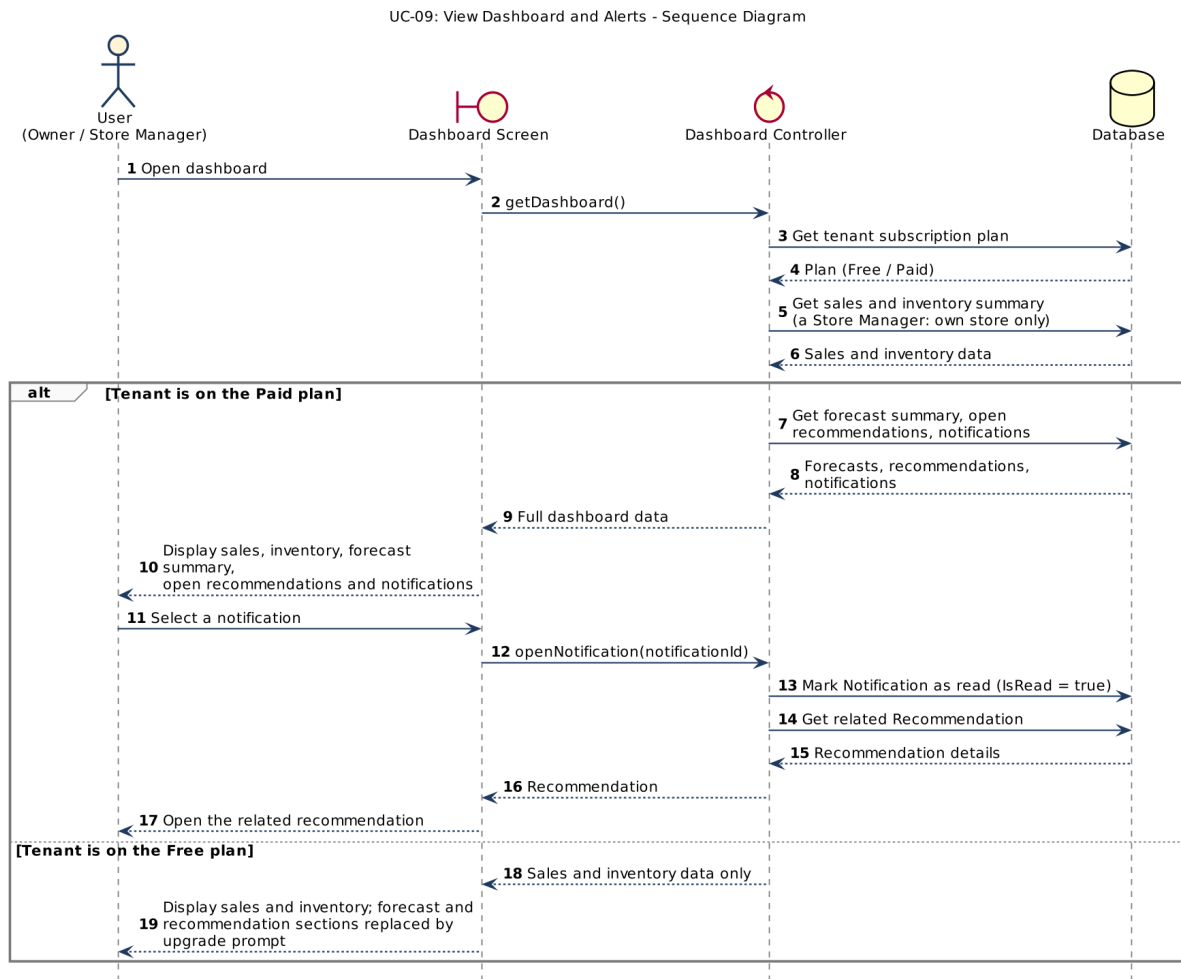


Figure 6.11: Sequence Diagram: View Dashboard and Alerts
 This figure shows the sequence diagram of UC-09 (View Dashboard and Alerts)

6.6.10 UC-10: View Demand Forecast

This diagram shows the interaction for the use case **View Demand Forecast** (UC-10).

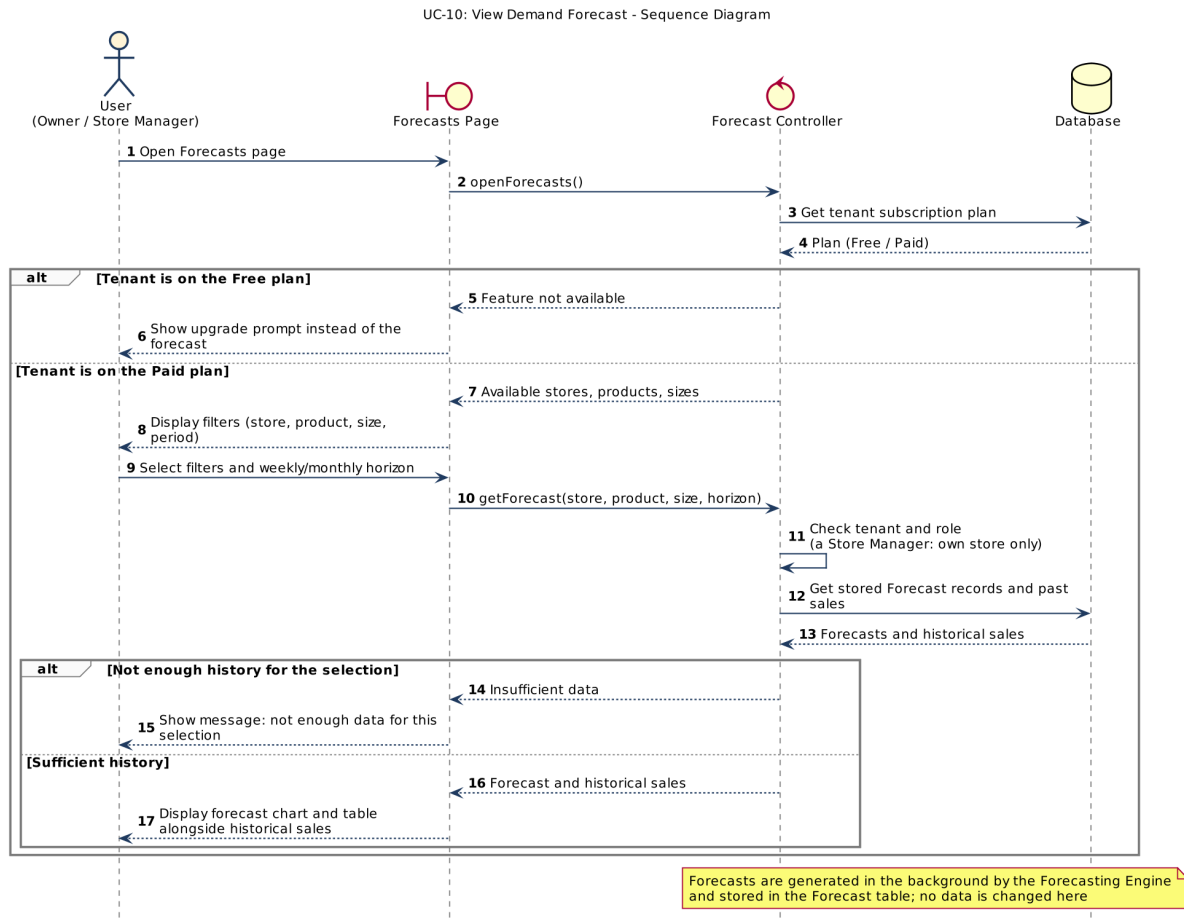


Figure 6.12: Sequence Diagram: View Demand Forecast
 This figure shows the sequence diagram of UC-10 (View Demand Forecast)

6.6.11 UC-11: View and Act on Redistribution Recommendation

This diagram shows the interaction for the use case **View and Act on Redistribution Recommendation** (UC-11).

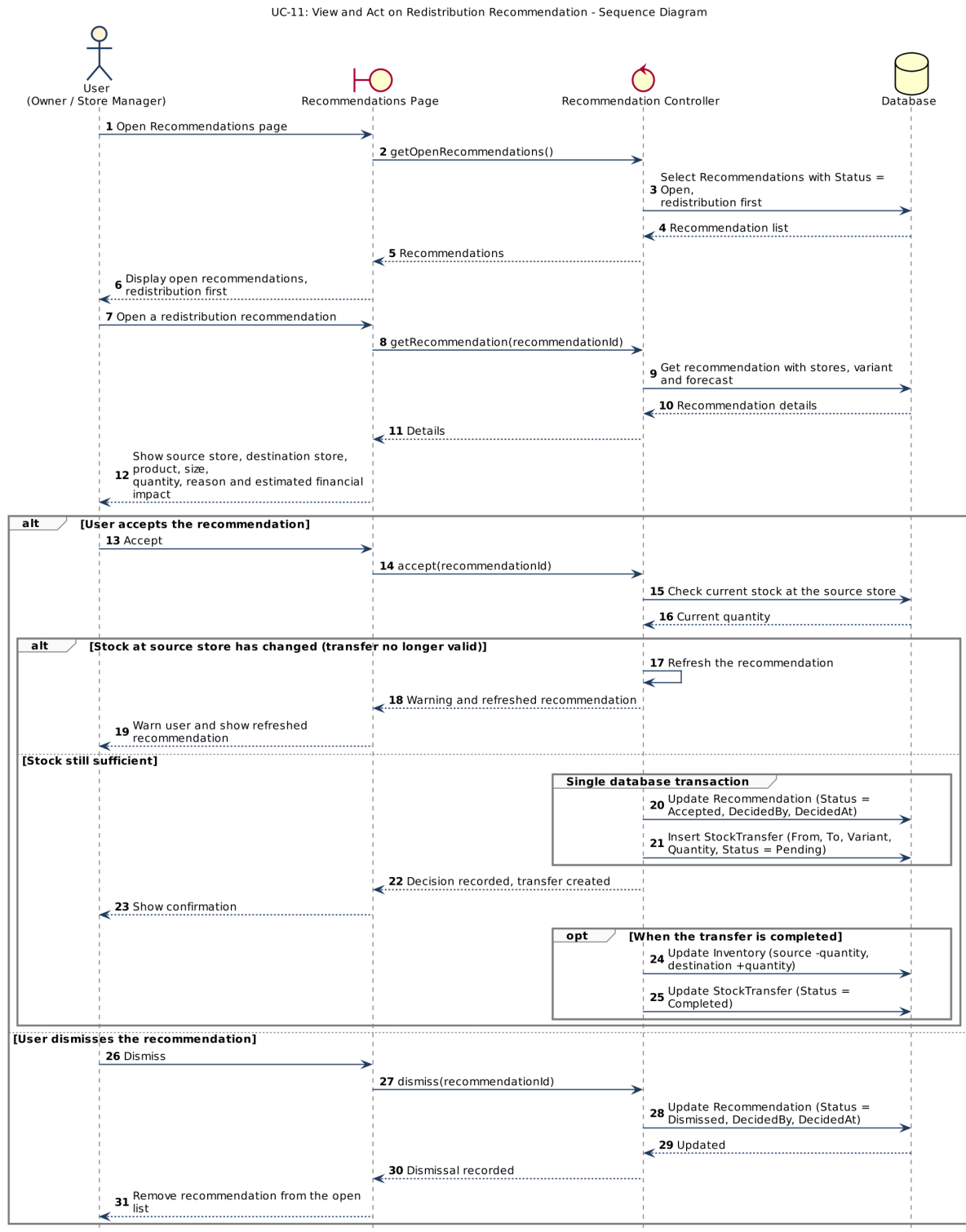


Figure 6.13: Sequence Diagram: View and Act on Redistribution Recommendation
 This figure shows the sequence diagram of UC-11 (View and Act on Redistribution Recommendation)

6.6.12 UC-12: View Markdown Recommendation

This diagram shows the interaction for the use case **View Markdown Recommendation** (UC-12).

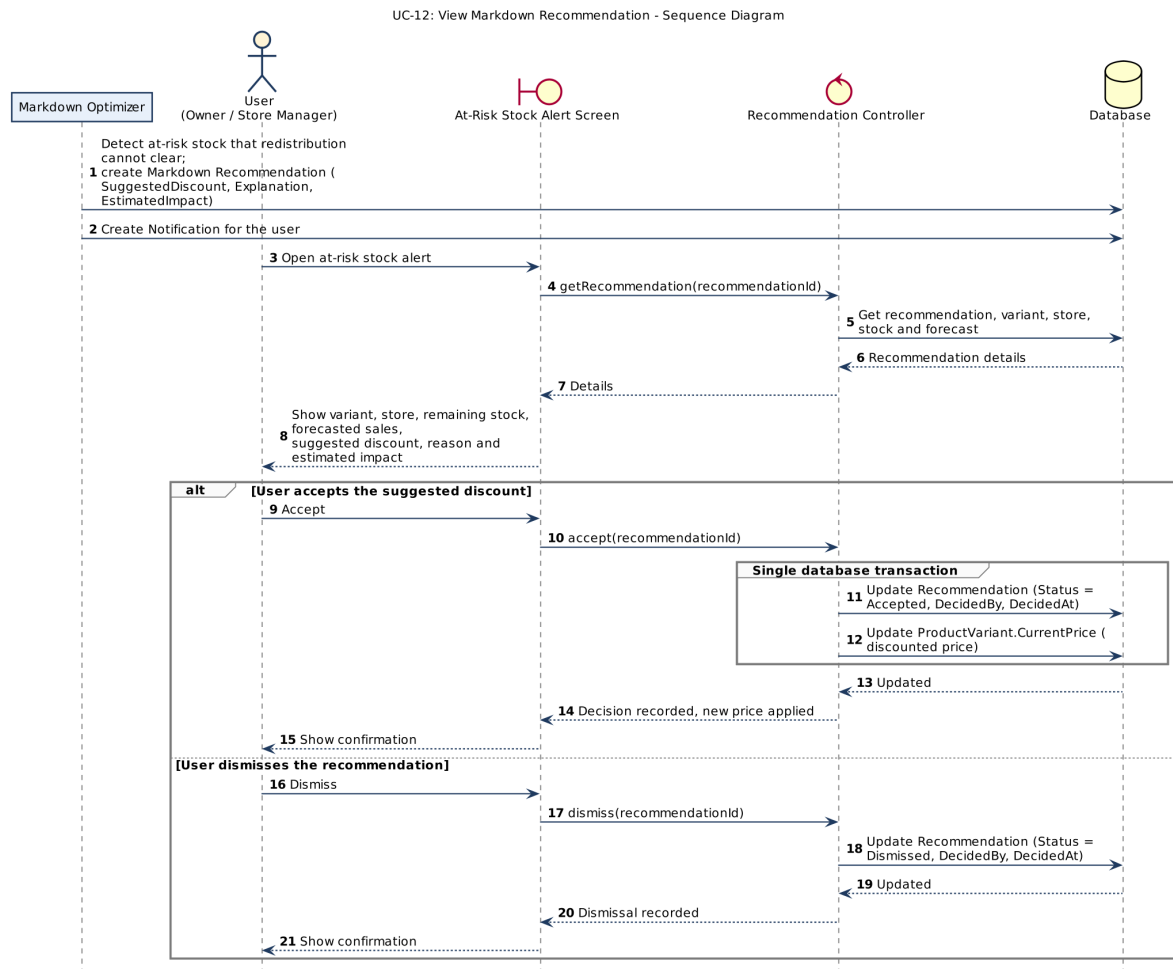


Figure 6.14: Sequence Diagram: View Markdown Recommendation
 This figure shows the sequence diagram of UC-12 (View Markdown Recommendation)

6.6.13 UC-13: View Reorder Recommendation

This diagram shows the interaction for the use case **View Reorder Recommendation** (UC-13).

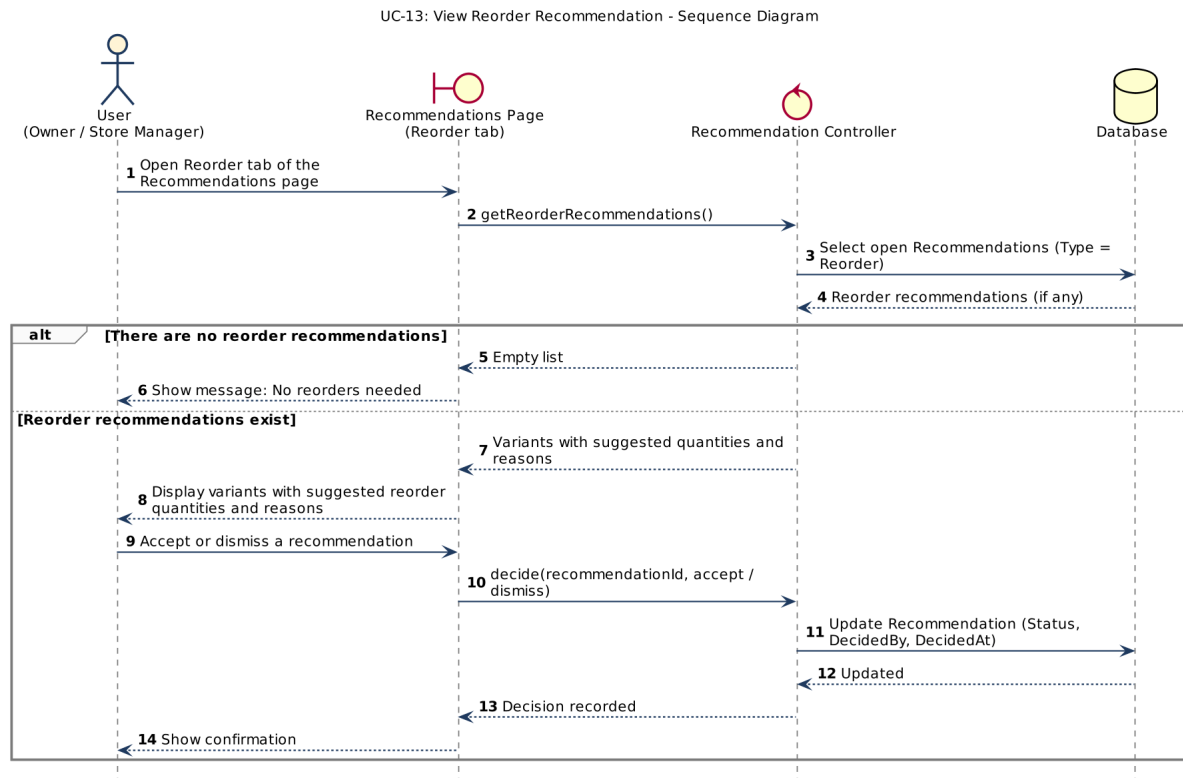


Figure 6.15: Sequence Diagram: View Reorder Recommendation
 This figure shows the sequence diagram of UC-13 (View Reorder Recommendation)

6.6.14 UC-14: Manage Subscription

This diagram shows the interaction for the use case **Manage Subscription** (UC-14).

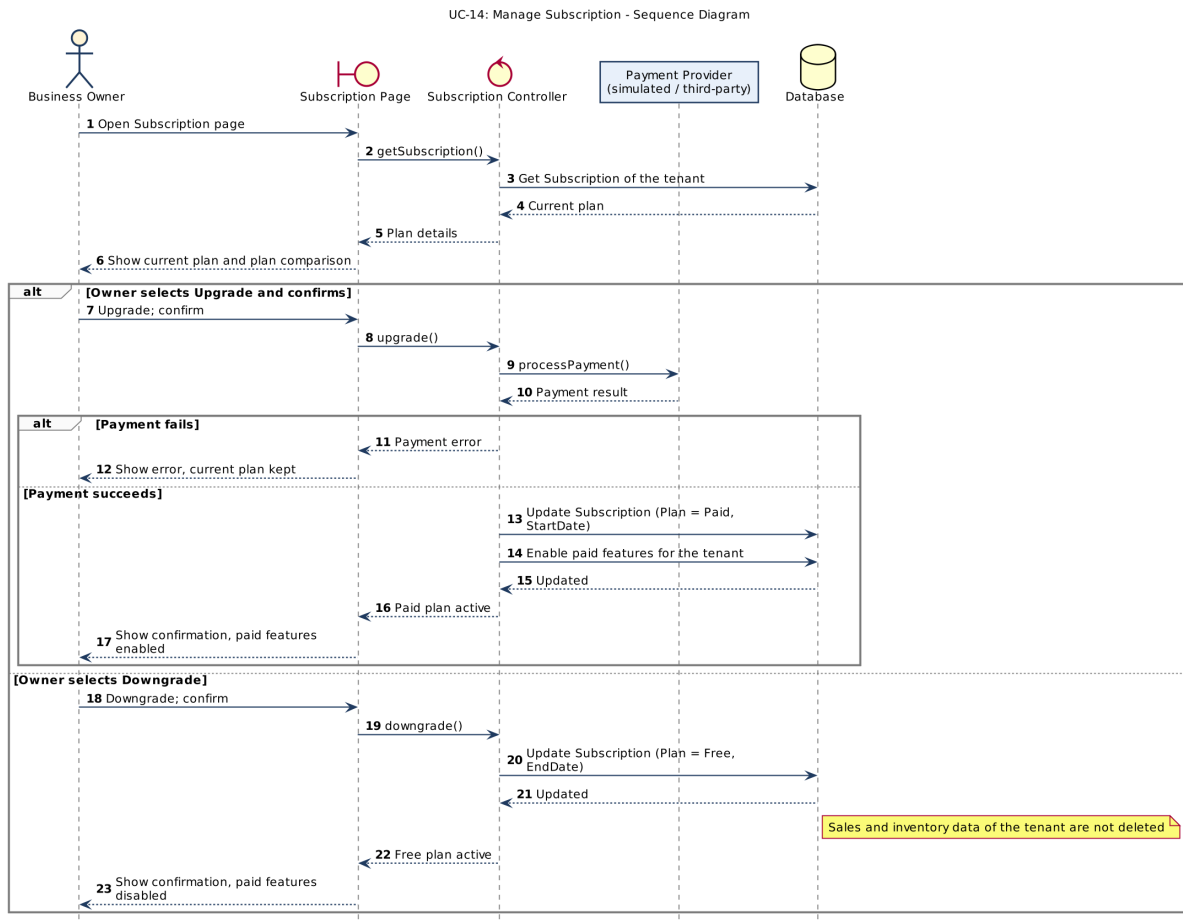


Figure 6.16: Sequence Diagram: Manage Subscription
 This figure shows the sequence diagram of UC-14 (Manage Subscription)

6.6.15 UC-15: Manage Tenants

This diagram shows the interaction for the use case **Manage Tenants** (UC-15).

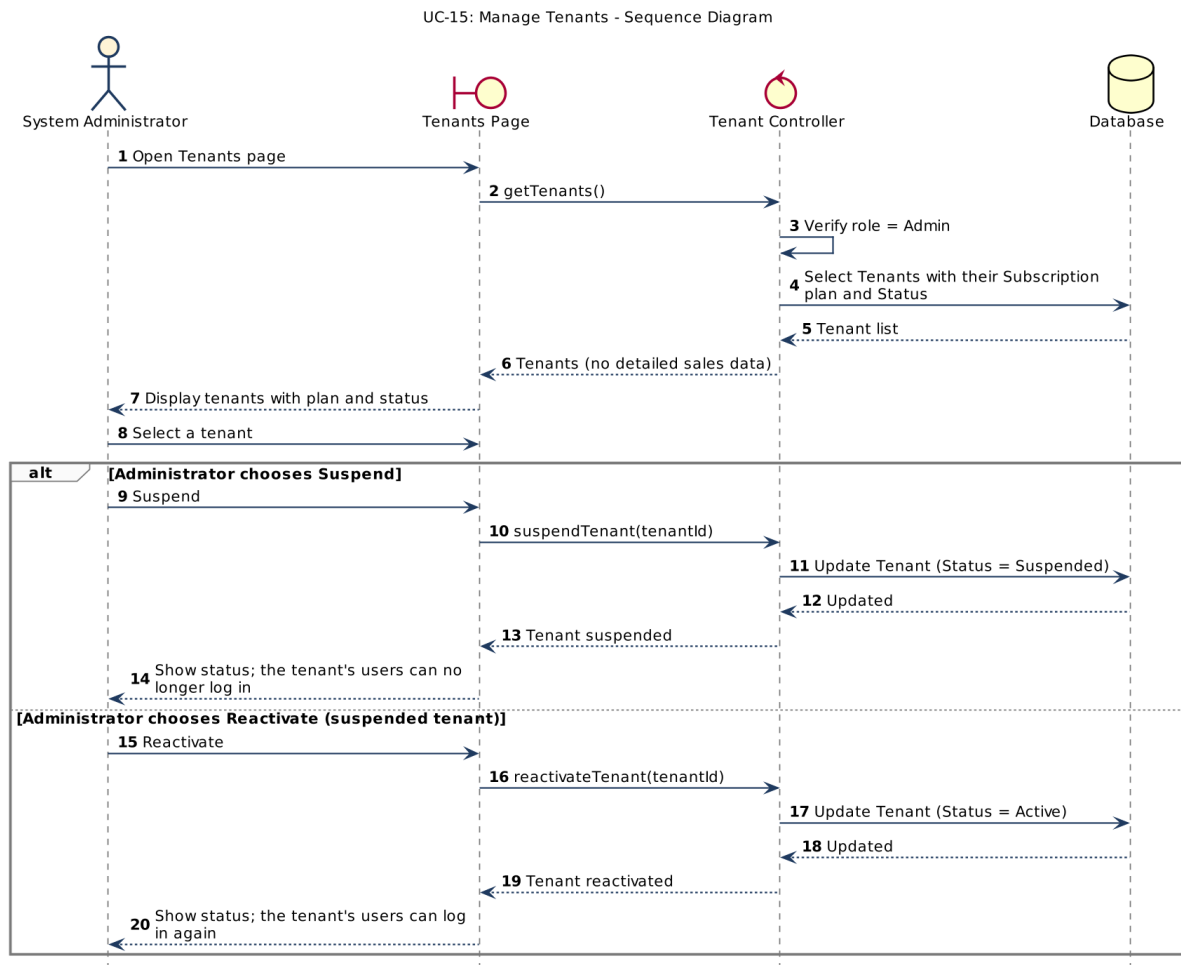


Figure 6.17: Sequence Diagram: Manage Tenants
 This figure shows the sequence diagram of UC-15 (Manage Tenants)

6.7 Policies and Tactics

While creating StockSense, the following policies and tactics will be applied:

6.7.1 Tools to be Used

A number of tools and libraries will be employed to develop StockSense. The most important ones are:

- The web dashboard and the POS screens will be created using React.
- The backend REST API will be implemented with Python and FastAPI or Django to handle authentication, tenant checks, and communication between the modules.
- All business data will be stored in PostgreSQL, and there will be a tenant identifier on each table that holds data for a business.
- The uploaded CSV and Excel files will be read, cleaned, and normalized using pandas and openpyxl.
- Data preparation, grouping of similar series, and calculation of error metrics will be supported using scikit-learn and NumPy, with LightGBM as the primary model for demand forecasting and simple statistical baseline models calculated using statsmodels.
- Version Control: Git and GitHub will be used to manage the software versions.
- Development: Visual Studio Code will be used for development.
- API testing: Postman will be used for API testing.
- Model experiments: Jupyter Notebook will be used for model experiments.

6.7.2 Coding Guidelines and Conventions

The backend will be written in Python, and the forecasting code in the same language, while the frontend will be implemented using a combination of JavaScript and React. The team will use a naming convention and write functions to be brief and easy to read, adding comments when the logic is not clear. The code will be kept organized by having each module reside in its own package. All changes will be checked by another team member before they are merged. Safe coding techniques will be observed throughout, such as storing passwords only in hashed form, parameterising database queries, and never storing secret keys in the repository.

6.7.3 User Interface

StockSense is designed for non-technical stock owners and managers and should be simple and easy to use, with minimal training required and clear navigation. A new user should be able to make a first sale without having to read any documentation. Forecasts will be presented as charts next to previous sales, and each recommendation is accompanied by a short explanation in plain language. Simple words will also be used to write error messages, e.g., if an uploaded file contains an error, this will be explained in a plain-language error message. The interface will be available on desktop computers, tablets, and phones.

6.7.4 Extensibility

The system will be designed in terms of modules, with each module having a specific function: POS and inventory, data ingestion, forecasting, recommendations, and markdown optimization. These modules will communicate through clear interfaces, which means that if one module is changed, the others do not need to change. The forecasting module will have a structure that allows various models to be tested and replaced without re-designing the application. The forecasting engine is general purpose and can be used by any business that stocks products with variants, sizes, or multiple stores, with new recommendation rules added as needed.

6.7.5 Tenant Data Isolation and Security

Each record in the system will be assigned to a tenant, and the backend will verify tenant ownership before each request reads or updates any data. Features will be accessible based on the role of the user, such that a store manager will only be able to view and update data of the store they have been assigned to. All communication will be via HTTPS. The system administrator can set up tenants and plans but will not be able to view the granular sales data unless required as part of support.

6.7.6 Data Validation

Uploaded sales files will be verified prior to being submitted to the forecasting pipeline. Invalid and incomplete rows will be flagged or rejected, and an upload report will inform the owner of what was accepted and what was rejected. In case an upload fails halfway, the data already stored will not be affected.

6.7.7 Explainable Recommendations

If a recommendation is made, the primary reasons for the recommendation (forecast, current stock, surplus/shortage, source branch) will be saved with it, including a financial impact estimate. This enables the owner to see the reason for each suggestion on the dashboard rather than an unexplained number.

6.7.8 System Testing

StockSense will have a testing policy that will involve unit testing, integration testing, and end-to-end testing. Individual tests will be executed on the parts of the system, including the validation rules and the inventory update after a POS sale. The API, database, and modules will be tested for correct integration, and the REST endpoints will also be tested using Postman. End-to-end tests will be completed following full user flows, such as registering a business, logging a sale, uploading a file, and taking action on a recommendation. Some tests will be specific to this project: tenant separation will be checked for data leakage, role-based access will be tested with each user type, and the upload pipeline will be tested with incomplete and poorly formed files. The forecasting models and the statistical baseline will be compared using MAPE and

RMSE on a data set that the models have not seen before. The performance requirements from the requirements chapter (e.g., saving a POS sale within 2 seconds, processing a file of 100,000 rows within 5 minutes) will also be tested prior to release.

6.7.9 Software Maintenance

The code will be maintained in a version-controlled repository on GitHub, and bugs and tasks will be tracked so that issues are not lost. Failed uploads, rejected records, and failed forecasting runs will be logged, making it easier to identify the cause. The models will be updated frequently with fresh data, as forecast accuracy diminishes when sales patterns change. The database will be backed up regularly to allow recovery of business data in the event of a failure. New team members will be able to grasp the system rapidly through up-to-date code and design documents.

6.7.10 Algorithms

The algorithms used in StockSense will be selected considering time efficiency and space efficiency, and will be verified by Big-O analysis to ensure that they scale with the number of tenants and the size of the sales history. Gradient-boosted trees (LightGBM), a method suitable for large retail data that performed very well in the M5 competition, will be used to forecast demand. One common model will be trained for each tenant across its products, sizes, and stores. A new tenant must first record or upload enough sales history, and until then no forecast is shown. Simple statistical models from statsmodels will be retained as a baseline to show whether the more complicated model is truly necessary. The recommendation algorithm will compare the forecast demand to the current stock of all branches for each variant. It will match branches with surplus to branches where a shortage is expected, then scan the remaining slow-moving stock to propose a markdown, and lastly suggest a reorder. The markdown logic will raise an alert when an item's selling speed indicates that the stock will not clear in time, so that corrective measures can be taken early. Most businesses do not have many branches, so this matching remains low cost. Large uploads and model training will be performed in the background to keep the system fast, the most recent forecasts will be saved rather than re-calculated when served, and the database will be indexed on the tenant identifier and the main keys.

6.8 Conclusion

In this chapter, the high level as well as the low level design of StockSense was explained. The proposed architecture separates out the tenant management, POS/inventory operations, data ingestion, forecasting, recommendations, and reporting on the dashboard into distinct modules. Tenant isolation, role-based access, data validation, background processing, and database indexing provide the foundation for a secure and scalable system. The design also supports the project's main decision flow: forecast demand, redistribute existing stock where possible, recommend markdowns for remaining slow-moving stock, and reorder only when necessary. The testing and maintenance strategy will be used to ensure the system is correct, performs well, is secure, and will be accurate before it is put into use.

Bibliography

- [1] P. K. Singh, Y. Gupta, N. Jha, and A. Rajan, "Fashion retail: Forecasting demand for new items," *arXiv preprint arXiv:1907.01960*, 2019. [Online]. Available: <https://arxiv.org/abs/1907.01960>
- [2] A. L. D. Loureiro, V. L. Miguéis, and L. F. M. da Silva, "Exploring the use of deep neural networks for sales forecasting in fashion retail," *Decision Support Systems*, vol. 114, pp. 81–93, 2018.
- [3] K. J. Ferreira, B. H. A. Lee, and D. Simchi-Levi, "Analytics for an online retailer: Demand forecasting and price optimization," *Manufacturing & Service Operations Management*, vol. 18, no. 1, pp. 69–88, 2016.
- [4] F. Caro and J. Gallien, "Clearance pricing optimization for a fast-fashion retailer," *Operations Research*, vol. 60, no. 6, pp. 1404–1422, 2012.
- [5] C. Paterson, G. Kiesmüller, R. Teunter, and K. Glazebrook, "Inventory models with lateral transshipments: A review," *European Journal of Operational Research*, vol. 210, no. 2, pp. 125–136, 2011.
- [6] K. Bandara, C. Bergmeir, and S. Smyl, "Forecasting across time series databases using recurrent neural networks on groups of similar series: A clustering approach," *Expert Systems with Applications*, vol. 140, p. 112896, 2020.
- [7] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022.
- [8] D. I. Purnamasari, V. A. Permadi, A. Saepudin, and R. P. Agusdin, "Demand forecasting for improved inventory management in small and medium-sized businesses," *Jurnal Nasional Pendidikan Teknik Informatika (JANAPATI)*, vol. 12, no. 1, pp. 56–66, 2023.
- [9] R. Fildes, S. Ma, and S. Kolassa, "Retail forecasting: Research and practice," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1283–1318, 2022.
- [10] P. Montero-Manso and R. J. Hyndman, "Principles and algorithms for forecasting groups of time series: Locality and globality," *International Journal of Forecasting*, vol. 37, no. 4, pp. 1632–1653, 2021.
- [11] M. L. Fisher and A. Raman, "Reducing the cost of demand uncertainty through accurate response to early sales," *Operations Research*, vol. 44, no. 1, pp. 87–99, 1996.
- [12] S. Thomassey, "Sales forecasts in clothing industry: The key success factor of the supply chain management," *International Journal of Production Economics*, vol. 128, no. 2, pp. 470–483, 2010.
- [13] M. Seyedan and F. Mafakheri, "Predictive big data analytics for supply chain demand forecasting: methods, applications, and research opportunities," *Journal of Big Data*, vol. 7, p. 53, 2020.
- [14] F. Petropoulos *et al.*, "Forecasting: theory and practice," *International Journal of Forecasting*, vol. 38, no. 3, pp. 705–871, 2022.
- [15] D. Bertsimas and N. Kallus, "From predictive to prescriptive analytics," *Management Science*, vol. 66, no. 3, pp. 1025–1044, 2020.
- [16] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

-
- [17] D. Corsten and T. Gruen, "Desperately seeking shelf availability: an examination of the extent, the causes, and the efforts to address retail out-of-stocks," *International Journal of Retail & Distribution Management*, vol. 31, no. 12, pp. 605–617, 2003.